

Building makemore Part 4： Becoming a Backprop Ninja

从零构建 makemore 第四部分：成为反向传播忍者（字幕版）｜手动推导梯度，彻底理解神经网络训练

时长 1:55:23 | 句数 1137 | 术语 41 | 章节 16

摘要

本讲共 16 章，从「自动微分 · 神经网络」到「训练循环 · PyTorch」。全文 1137 句、1136 句已译成中文，标注了 42 个 AI 术语（主要集中在神经网络基础、数学与微积分、反向传播与梯度）。

16 chapters, 1137 sentences (1136 translated), 42 AI terms annotated. Built fully offline with opus-mt + local glossary.

自动微分 · 神经网络 00:00

00:00 Hi everyone. So today we are once again continuing our implementation of Make More.
大家好,今天我们再次继续 执行《创造更多》

implementation
实现 实现
代码与工程

00:05 Now so far we've come up to here, multi- layer perceptrons, and our neural net looked like this,
现在我们走到这里,多层感官, 我们的神经网看起来像这样,

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

neural net
神经网络 神经网络
神经网络基础

00:10 and we were implementing this over the last few lectures. Now I'm sure everyone is very excited
并且我们在过去几场讲座里 执行这个计划。现在我相信每个人都非常兴奋

00:14 to go into recurrent neural networks and all of their variants and how they work,
进入经常神经网络 以及所有变异体 以及如何运作

00:18 and the diagrams look cool and it's very exciting and interesting and we're going to get a better
图表看起来很酷,非常令人兴奋和有趣 我们会得到更好的

00:21 result. But unfortunately I think we have to remain here for one more lecture. And the reason
。但不幸的是,我认为我们必须留下来再做一次演讲。

00:27 for that is we've already trained this multi- layer perceptron, right, and we are getting pretty good
因为我们已经训练了这个多层的感官,对,而且我们越来越好

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

00:32 loss . And I think we have a pretty decent understanding of the architecture and how it works.
我认为我们对建筑及其运作方式 有着相当的正确理解

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

00:36 But the line of code here that I take an issue with is here, lost of backward. That is we are
但这里我所要处理的代码线 就是在这里,失去后退。

00:42 taking PyTorch autograd and using it to calculate all of our gradients along the way.
使用平托尔奇自化法 计算我们沿途的所有梯度

pytorch
PyTorch PyTorch
代码与工程

autograd
自动微分 自动微分
反向传播与梯度

自动微分 · 反向传播 00:47

00:47 And I would like to remove the use of lost of backward, and I would like us to write our
我想取消对落后的失落的利用, 我希望我们写下

00:51 backward pass manually on the level of tensors. And I think that this is a very useful exercise
以 10 的 水平手动向后过后传到 。 我认为这是一个非常有用的练习

> **backward pass**
> 反向传播 反向传播
> 神经网络基础

00:56 for the following reasons. I actually have an entire blog post on this topic, but I like to
原因如下。 我实际上有整个博客文章,

01:02 call back propagation a leaky abstraction. And what I mean by that is back propagation doesn't
即是反向传播

> **back propagation**
> 反向传播 反向传播
> 神经网络基础

01:08 just make your neural networks just work magically. It's not the case that you can just stack up
让你的神经网络奇迹般地运转起来

01:12 arbitrary Lego blocks of differentiable functions and just cross your fingers and back propagate
任意任意的乐高区块,具有不同功能,只需交叉手指和反向传播

01:17 and everything is great. Things don't just work automatically. It is a leaky abstraction in
一切都很好,事情不只是自动运作,而是在

01:22 the sense that you can shoot yourself in the foot if you do not understand its internals.
感觉到如果你不理解它的内脏 你可以射自己的脚

01:26 It will magically not work or not work optimally. And you will need to understand how it works under
它将无法发挥神奇的功效,也不能发挥最佳效果。你需要了解它是如何运作的。

01:32 the hood if you're hoping to debug it. And if you are hoping to address it in your neural net.
如果你想调试它,如果你希望 解决在神经网。

> **neural net**
> 神经网络 神经网络
> 神经网络基础

01:37 So this blog post here from a while ago goes into some of those examples. So for example,
本文博客文章是不久前在这里发来的,

01:42 we've already covered them. Some of them already, for example, the flat tails of these functions
其中一些已经覆盖了它们,例如这些函数的平面尾巴

01:48 and how you do not want to saturate them too much because your gradients will die.
以及你们怎么不希望他们太饱和,因为你们的梯度将死亡。

01:53 The case of dead neurons, which I've already covered as well. The case of exploding or vanishing
死亡神经元的案例,我已经也报道过。爆炸或消失的案例

01:58 gradients in the case of recurrent neural networks, which we are about to cover.
经常神经网络的梯度,我们即将覆盖。

02:02 And then also you will often come across some examples in the wild. This is a snippet that I
然后你也会经常在野外遇到一些例子。这是我的一小块碎片。

02:08 found in a random code base on the internet where they actually have like a very subtle but
在一个随机代码基中找到的 在互联网上,他们实际上有 像一个非常微妙的 但是

02:13 pretty major bug in their implementation. And the bug points at the fact that the author
执行中存在非常大错误。 而错误则指出作者

implementation
实现 实现
代码与工程

02:19 of this code does not actually understand back propagation. So they're trying to do here
这个代码实际上不理解回传。 所以他们试图在这里做

back propagation
反向传播 反向传播
神经网络基础

02:22 is they're trying to clip the loss at a certain maximum value. But actually what they're trying to
试图以一定的最大价值来计算损失。

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

02:27 do is they're trying to clip the gradients to have a maximum value instead of trying to
他们是否试图剪切梯度 以便有一个最大值而不是尝试

02:31 clip the loss at a maximum value. And indirectly, they're basically causing some of the outliers
间接地,它们基本上导致一些外线

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

02:37 to be actually ignored. Because when you clip a loss of an outlier, you are setting its gradient
被实际上忽略。 因为当您剪切一个外线损失时, 您正在设置其梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

02:43 to zero. And so have a look through this and read through it. But there's basically a
零到零。所以仔细看看这个,读过它。但基本上有一个

02:49 bunch of subtle issues that you're going to avoid if you actually know what you're doing.
如果你真的知道自己在做什么 就会避免一些微妙的问题

02:52 And that's why I don't think it's the case that because PyTorch or other frameworks offer
这就是为什么我不认为 之所以如此,是因为PyTorrch或其他框架

pytorch
PyTorch PyTorch
代码与工程

02:56 autograd, it is okay for us to ignore how it works. Now, we've actually already covered
自动升级,我们忽略它是如何运作的,这是可以的。现在,我们已经覆盖了

autograd
自动微分 自动微分
反向传播与梯度

03:02 autograd and we wrote micrograd. But micrograd was an autograd engine only on the level of
自动升级,我们写了微升。但是微升只是个自动升级引擎

autograd
自动微分 自动微分
反向传播与梯度

micrograd
micrograd 自动微分库 micrograd 自动微分库
神经网络基础

03:08 individual scalars. So the atoms were single individual numbers. And, you know, I don't
原子是单个数字。你知道,我不

03:13 think it's enough. And I'd like us to basically think about back propagation on
我想我们基本上考虑回溯传播

back propagation
反向传播 反向传播
神经网络基础

03:16 level of tensors as well. And so in a summary, I think it's a good exercise.
也存在几何水平。所以在总结中, 我认为这是一个很好的练习。

03:20 I think it is very, very valuable. You're going to become better at debugging neural networks
这会变得更好调试神经网络

03:25 and making sure that you understand what you're doing. It is going to make everything fully
并且确保你明白自己在做什么。它将使一切变得充分

03:29 explicit. So you're not going to be nervous about what is hidden away from you. And basically,
很明显,所以你不会对隐藏在你身边的东西感到紧张。

03:33 in general, we're going to emerge stronger. And so let's get into it. A bit of a fun historical
总的来说,我们将会变得更强大。让我们来讨论一下。一个有趣的历史

03:38 note here is that today, writing your backward pass by hand and manually is not recommended
请注意,今天,用手写后方通行证和手工写字不推荐

backward pass
反向传播 反向传播
神经网络基础

03:43 and no one does it except for the purposes of exercise. But about 10 years ago in deep
10年前,在深深的黑暗中

03:48 learning, this was fairly standard and in fact pervasive. So at the time, everyone used to write
学习,这是相当标准且实际上普遍存在的。所以当时每个人都写过

03:52 their own backward pass by hand manually, including myself. And it's just what you would do. So we
包括我自己在内,你也会这样做。所以我们

backward pass
反向传播 反向传播
神经网络基础

03:58 used to write backward pass by hand. And now everyone just called lost that backward. We've
现在大家都被叫为"失去后退"

backward pass
反向传播 反向传播
神经网络基础

损失函数 · 训练循环 04:02

04:02 lost something. I want to give you a few examples of this. So here's a 2006 paper from
失去的东西。我想举几个例子。这里是2006年的论文

04:09 Jeff Hinton and Ruslan Slakhtinov in science that was influential at the time. And this
Jeff Hinton和Ruslan Slakhtinov 研究当时有影响力的科学。这个

04:15 was training some architectures called restricted Boltzmann machines. And basically, it's an
训练一些建筑 叫做限制的波尔兹曼机器

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

04:20 autoencoder trained here. And this is from roughly 2010. I had a library for training
自动编码器在这里受训。这是大约2010年的,我有一个图书馆用于培训

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

04:26 restricted Boltzmann machines. And this was at the time written in Matlab. So Python was not
限制使用波尔兹曼机器。这是在当时写在Matlab的。所以Python不是

04:32 used for deep learning pervasively. It was all Matlab. And Matlab was this scientific
用来进行深层学习的 广泛。这是所有马特拉布。而马特拉布是这个科学

04:36 computing package that everyone would use. So we would write Matlab, which is barely a programming
所有人都会使用的计算计算机套件。 所以我们写Matlab, 这只不过是个程序

04:42 language as well. But it had a very convenient tensor class. And it was this computing environment
语言也是。但它有一个非常方便的 发压类。它就是这个计算环境

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

04:48 and you would run here. It would all run on the CPU, of course, but you would have very nice
你会跑到这里。当然,它会全部运行在CPU上,但你会非常漂亮

04:52 plots to go with it and a built in debugger. And it was pretty nice. Now the code in this
和一个在调试器中建的调试器。它非常漂亮。现在这个代码

04:57 package in 2010 that I wrote for fitting restricted Boltzmann machines to a large extent
2010年我写了一篇关于 在很大程度上安装限制使用波尔兹曼机器的

05:03 is recognizable. But I wanted to show you how you would, well, I'm creating the data
看得出来 但我想告诉你 你会怎么做

05:07 and the XY batches. I'm initializing the neural net. So it's got weights and biases just like
和XY批量。我正在初始开始神经网。所以它有重量和偏向 就像

neural net
神经网络 神经网络
神经网络基础

05:12 we're used to. And then this is the training loop where we actually do the forward pass.
我们习惯了,然后这是训练循环,我们实际上就是在那里 做前方传球。

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

forward pass
前向传播 前向传播
神经网络基础

05:17 And then here, at this time, didn't even necessarily use back propagation to train neural
此时此刻,这里甚至没有用回传来训练神经

back propagation
反向传播 反向传播
神经网络基础

05:21 networks. So this in particular implements contrastive divergence, which estimates
因此,这尤其造成了截然不同的差异,估计

05:27 a gradient. And then here we take that gradient and use it for a parameter update along
渐变。然后我们在这里取出梯度,然后用它来更新参数

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

05:32 the lines that we're used to. Yeah, here. But you can see that basically people are meddling
是的,这里,但你可以看到 基本上人们在干预

05:38 with these gradients directly and inline and themselves. It wasn't that common to use an
和这些梯度直接、内线和内部的梯度, 以及它们本身。

05:42 autograd engine. Here's one more example from a paper of mine from 2014 called the fragment
自动升级引擎。 下面是2014年的一张矿纸上的另一个例子, 叫做碎片

autograd
自动微分 自动微分
反向传播与梯度

05:48 embeddings. And here what I was doing is I was aligning images and text. And so it's kind
嵌入。我所做的是,我正在对图像和文字进行调整。所以它很温柔

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

05:54 of like a clip if you're familiar with it. But instead of working on the level of
而不是在

05:57 entire images and entire sentences, it was working on the level of individual objects
整个图像和整句句子, 它正在研究单个物体的级别

06:01 and little pieces of sentences. And I was embedding them and then calculating a very much
和几句小句子。我把它们嵌入,然后计算了很多

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

06:05 like a clip like loss. And I dug up the code from 2014 of how I implemented this.
我从2014年开始就找出了密码 说明我是如何执行的

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

06:10 And it was already in NumPy and Python. And here I'm implementing the cost function.
已经到了NumPy和Python了,我在这里执行成本功能

06:16 And it was standard to implement not just the cost, but also the backward pass manually.
不仅执行成本标准,而且人工操作后传也是标准。

backward pass
反向传播 反向传播
神经网络基础

06:21 So here I'm calculating the image embeddings, sentence embeddings,
所以我在计算图像嵌入 句子嵌入

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

06:25 the loss function. I calculate the scores. This is the loss function. And then once I have
损失函数。 我计算分数。 这是损失函数。 一旦我得到

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

06:31 the loss function, I do the backward pass right here. So I backward through the loss function
损失功能,我在这里做后向传递。所以我从损失功能中退过

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

backward pass
反向传播 反向传播
神经网络基础

06:36 and through the neural net and I append regularization. So everything was done by hand manually. And you
并且通过神经网络,我附加了正常化。所以一切都是手工完成的。你也是

neural net
神经网络 神经网络
神经网络基础

06:42 were just right out the backward pass. And then you would use a gradient checker to make sure
正好在后退路口 然后你用梯度检查器确保

backward pass
反向传播 反向传播
神经网络基础

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

06:46 that your numerical estimate of the gradient agrees with the one you calculated during
计算时计算的数字一致

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

06:49 the back propagation. So this was very standard for a long time. But today, of course, it is
后背传播。所以这在很长一段时间里是非常标准的, 但是今天,当然,它

back propagation
反向传播 反向传播
神经网络基础

06:53 standard to use an autograd engine. But it was definitely useful. And I think people sort
使用自动格化引擎的标准。 但是它绝对有用。 我认为人们会排序

autograd
自动微分 自动微分
反向传播与梯度

06:59 of understood how these neural networks work on a very intuitive level. And so I think it's a good
理解这些神经网络是如何在非常直觉的层面上运作的。 所以我觉得这是一个很好的

数据集 · 损失函数 07:03

07:03 exercise again. And this is where we want to be. Okay, so just as a reminder from our previous lecture,
练习。这是我们想要的。好的,正如我们上次演讲中的提醒:

07:07 this is the Jupyter notebook that we implemented at the time. And we're going to keep everything
这是我们当时使用的 尤比特笔记本

07:12 the same. So we're still going to have a two layer multilayer perceptron with a batch
相同。所以我们仍会有一个两层多层的多层透视器,配有批量

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

07:16 normalization layer. So the forward pass will be basically identical to this lecture.
正常的层。 因此前进通道将基本上和这个演讲相同。

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

forward pass
前向传播 前向传播
神经网络基础

07:20 But here we're going to get rid of loss.backward. And instead, we're going to write the
但是我们在这里要摆脱损失,退后,相反,我们要写

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

07:24 backward pass manually. Now, here's the starter code for this lecture. We are becoming a
手动向后后过后过,这是这个讲座的起始代码。我们正成为

backward pass
反向传播 反向传播
神经网络基础

07:28 backprop ninja in this notebook. And the first few cells here are identical to what we are used to.
和我们习惯的相同

backprop
反向传播 反向传播
反向传播与梯度

07:35 So we are doing some imports, loading the data set and processing the data set. None of this changed.
所以我们正在做一些进口, 装载数据集和处理数据集。这些都没有改变。

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

07:41 Now here I'm introducing a utility function that we're going to use later to compare the
现在我介绍一个实用功能 稍后我们用它来比较

07:45 gradients. So in particular, we are going to have the gradients that we estimate manually
梯度。 因此,特别是,我们将有我们手工估算的梯度

07:49 ourselves. And we're going to have gradients that PyTorch calculates. And we're going to be
并且我们将会有PyTork计算出的梯度。

pytorch
PyTorch PyTorch
代码与工程

07:53 checking for correctness, assuming of course that PyTorch is correct. Then here we have the
假设皮托尔奇是对的

pytorch
PyTorch PyTorch
代码与工程

07:59 initialization that we are quite used to. So we have our embedding table for the characters,
我们非常习惯于初始化。所以我们有了嵌入表格 来显示字符,

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

08:04 the first layer, second layer and a batch normalization in between. And here's where we
中间的批量正常化。这里就是我们

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

08:09 create all the parameters. Now you will note that I changed the initialization a little bit
创建所有参数。 现在, 您将注意到我对初始化稍稍修改

08:14 to be small numbers. So normally you would set the biases to be all zero. Here I'm
通常你会把偏差设置为零。我在这里

08:18 setting them to be small random numbers. And I'm doing this because if your variables are
设置它们为小随机数字。 我这样做是因为如果变量是

08:23 initialized to exactly zero, sometimes what can happen is that can mask an incorrect
初始化为零, 有时会发生的情况是, 它可以隐藏错误

08:27 implementation of a gradient. Because when everything is zero, it sort of like simplifies
渐渐度的安装。 因为当一切都为零时, 它就像简化

implementation
实现 实现
代码与工程

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

08:33 and gives you a much simpler expression of the gradient than you would otherwise get.
并且使你对于梯度的表达比以前更简单。

expression
表达式 表达式
神经网络基础

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

08:36 And so by making it small numbers, I'm trying to unmask those potential errors
所以通过做小数字,我试图揭开这些潜在的错误

08:41 in these calculations. You also notice that I'm using B1 in the first layer. I'm using a
计算。 您也注意到我在第一层使用 B1 。

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

08:47 bias despite batch normalization right afterwards. So this would typically not be what you do because
通常不会是你们做的,因为

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

08:53 we talked about the fact that you don't need a bias. But I'm doing this here just for fun.
我们谈过你不需要偏见的事实 但我这么做只是为了好玩

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

08:57 Because we're going to have a gradient with respect to it and we can check that we are
因为我们将会有一个梯度 与此有关 我们可以检查我们

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

09:00 still calculating it correctly even though this bias is as pervious. So here I'm calculating a
即使这个偏差是显而易见的,我仍然在正确计算。

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

09:06 single batch. And then here I am doing a forward pass. Now you'll notice that the
单批单批。然后,我在这里做一个前方传过去。现在你会注意到

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

forward pass
前向传播 前向传播
神经网络基础

09:11 forward pass is significantly expanded from what we are used to. Here the forward pass was just
前方的通行证大大扩展了我们过去的做法。

forward pass
前向传播 前向传播
神经网络基础

09:16 here. Now the reason that the forward pass is longer is for two reasons. Number one, here we just had
。 现在, 向前通行证更长的原因是有两个原因。 第一, 我们刚刚在这里

forward pass
前向传播 前向传播
神经网络基础

09:22 an F dot cross entropy. But here I am bringing back a explicit implementation of the loss function.
一种F点交叉星体。但我在这里带回了明确执行损失功能的信号。

cross entropy
交叉熵 交叉熵
数学与微积分

implementation
实现 实现
代码与工程

09:28 And number two, I've broken up the implementation into manageable chunks. So we have a lot more
第二,我把执行程序分成了可控部分。所以我们还有很多

implementation
实现 实现
代码与工程

09:35 intermediate tensors along the way in the forward pass. And that's because we are about to
中间的10个数沿着前方路道前进。那是因为我们即将

forward pass
前向传播 前向传播
神经网络基础

09:39 go backwards and calculate the gradients in this back propagation from the bottom to the top.
向后向后,计算从下到上向上传播的此后面的梯度。

back propagation
反向传播 反向传播
神经网络基础

09:45 So we're going to go upwards. And just like we have, for example, the log props tensor in a forward
所以我们要往上走,就像我们一样, 木道的推进力在向前

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

09:50 pass, in the backward pass, we're going to have a d log props, which is going to store the derivative
后过,在后过,我们有一个D对道道具, 它会存储衍生物

backward pass
反向传播 反向传播
神经网络基础

derivative
导数 导数
反向传播与梯度

09:55 of the loss with respect to the log props tensor. And so we're going to be pre-pending d
原木道的损耗 压力。所以我们将要预先准备 d

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

10:00 to every one of these tensors and calculating it along the way of this back propagation.
每一个这些发热器,并计算它 沿这条路的回传。

back propagation
反向传播 反向传播
神经网络基础

10:05 So as an example, we have a b and raw here. We're going to be calculating a d, b and raw.
举例来说,我们这里有一个b和生。我们将计算d、b和生的。

10:10 So here I'm telling PyTorch that we want to retain the grad of all these intermediate
所以我在这里告诉皮托尔奇 我们想保留这些中间

pytorch
PyTorch PyTorch
代码与工程

10:15 values because here in exercise one, we're going to calculate the backward pass. So we're going to
值,因为在这里练习一中, 我们计算后向的过错。 所以我们要

backward pass
反向传播 反向传播
神经网络基础

10:20 calculate all these d variables and use the cmp function I've introduced above to check our
计算所有这些 d 变量, 并使用我在上面引入的 cmp 函数来检查

10:25 correctness with respect to what PyTorch is telling us. So this is going to be exercise one,
俾托尔奇告诉我们的,是真实的。这是一次锻炼,

pytorch
PyTorch PyTorch
代码与工程

10:31 where we sort of back propagate through this entire graph. Now just to give you a very quick
我们回到了这里,通过整个图象传播。现在,给你一个非常快速的

10:36 preview of what's going to happen in exercise two and below, here we have fully broken up
练习二和二的预览 以及下面的预览 我们完全分开了

10:41 the loss and back propagated through it manually in all the little atomic pieces that make it up.
损失和回路通过它人工传播 在所有小原子碎片中 组成它。

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

10:47 But here we're going to collapse the loss into a single cross entropy call.
但是我们在这里要把损失 变成一个十字的星号

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

cross entropy
交叉熵 交叉熵
数学与微积分

10:51 And instead we're going to analytically derive using math and paper and pencil the gradient of
而不是我们用数学和纸学 来分析出

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

10:58 the loss with respect to the logits. And instead of back propagating through all of its little
而不是在它所有的小小的角落里 反倒传播

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

11:02 chunks one at a time, we're just going to analytically derive what that gradient is
一次一个块块, 我们只是要分析得出什么是梯度

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

11:06 and we're going to implement that, which is much more efficient, as we'll see in a bit.
我们将执行这个, 效率更高,我们将会看到。

11:10 Then we're going to do the exact same thing for batch normalization. So instead of breaking
然后我们做同样的事情 批量正常化。所以,而不是打破

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

11:14 up batch charming to all the little tiny components, we're going to use pen and paper and mathematics
整批精采的批量 对所有小小部件的迷人, 我们将使用笔和纸 数学

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

11:20 and calculus to derive the gradient through the batch, batch from layer. So we're going to
和计算,通过分批、分层和分层来产生梯度。所以我们要去

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

11:26 calculate the backward pass through batch from layer in a much more efficient expression,
以效率更高的表达式计算从层到下端的批次后向传递,

backward pass
反向传播 反向传播
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

11:29 instead of back propagating through all of its little pieces independently.
而不是在背后 独自传播它的所有小碎片

11:33 So it's going to be exercise three. And then in exercise four, we're going to put it all
将进行三场运动,然后在四场运动中,我们把它全部

11:38 together. And this is the full code of training this two layer MLP. And we're going to basically
并且我们基本上要

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

11:43 insert our manual back prop, we're going to take out lost the backward, and you will basically see
插入我们的手动后背道具, 我们将拿掉失去的后向, 你会看到

11:49 that you can get all the same results using fully your own code. And the only thing we're
完全使用自己的代码,你就能得到所有相同的结果。 而我们是唯一

11:55 using from pytorch is the torch dot tensor to make the calculations efficient. But otherwise,
使用燃烧火是火炬点柱的电压 使计算效率有效。但是,否则,

pytorch
PyTorch PyTorch
代码与工程

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

12:01 you will understand fully what it means to forward and backward neural net and train it.
并且训练它。

neural net
神经网络 神经网络
神经网络基础

12:05 And I think that will be awesome. So let's get to it. Okay, so I ran all the cells of this notebook
我想那会很棒。让我们开始吧。好吧,我查了这个笔记本的所有细胞

12:10 all the way up to here. And I'm going to erase this and I'm going to start implementing backward pass
我要抹去这个,我要开始执行后发通行证

backward pass
反向传播 反向传播
神经网络基础

12:15 starting with the log problems. So we want to understand what should go here to calculate
开始于日志问题。 所以我们想要了解要在这里计算什么

12:20 the gradient of the loss with respect to all the elements of the log problems tensor.
与日志问题的所有内容有关的损失梯度。

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

12:25 Now I'm going to give away the answer here, but I wanted to put a quick note here that
现在我要在这里给出答案 但我想在这里简短地说一句

12:29 I think would be most pedagogically useful for you is to actually go into the description of
我认为,在教学上对你最有用的是 实际去描述

12:34 this video and find the link to this Jupyter notebook. You can find it both on GitHub, but you
找到与这个Jupyter笔记本的链接。

12:38 can also find Google collab of it. So you don't have to install anything, you'll just go to a website
也可以找到谷歌的连锁合机。所以你不必安装任何东西, 你只需去一个网站

12:42 on Google collab. And you can try to implement these derivatives or gradients yourself. And then
你可以尝试自己执行这些衍生物或梯度。然后

12:48 if you are not able to come to my video and see me do it. And so work in tandem and try it first
如果你不能来看我的视频,看不看我做。所以一起工作,先试一下

12:54 yourself and then see me give away the answer. And I think that will be most valuable to you.
并且让我把答案给自己, 我认为这对你来说是最重要的。

12:58 And that's how I recommend you go through this lecture. So we are starting here with
所以我建议你这样来讲课

13:02 the log props. Now the log props will hold the derivative of the loss with respect to all the
现在日志道具将持有损失的衍生物

derivative
导数 导数
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

批次大小 · 损失 13:08

13:08 elements of log props. What is inside log blocks, the shape of this is 32 by 27. So it's not going
日志 道具元素。什么是圆木块中的, 这个形状是 32x27, 所以它不会去

13:17 to surprise you that the log props should also be an array of size 32 by 27, because we want the
来给你惊喜,因为我们需要的

13:22 derivative loss with respect to all of its elements. So the sizes of those are always going to be
其所有元素的衍生衍生物损失。 因此这些元素的大小总是

derivative
导数 导数
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

13:27 equal. Now how does log props influence the loss? Loss is negative log props indexed with
等值。 现在日对道

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

13:38 range of n and yb and then the mean of that. Now just as a reminder, yb is just basically an array
n 和 yb 区域,然后是该区域平均值。现在,作为提醒,yb 基本上是一个阵列

13:47 of all the correct indices. So what we're doing here is we're taking the log props array of size
正确的指数。所以我们在这里做的是 我们用的是 大小的日志道具阵列

13:54 32 by 27. And then we are going in every single row and in each row we are plugging out the index
32乘27,然后我们进入每一行 每行每行每行 插入索引

14:05 8 and then 14 and 15 and so on. So we're going down the rows, that's the iterator range of n,
8和14,15等等。所以我们往下走, 这是循环范围 n,

14:10 and then we are always plugging out the index at the column specified by this tensor yb. So in
然后我们总是在 这个 发光 yb 指定的列中插入索引。 所以, 在

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

14:16 the 0th row, we are taking the 8th column. In the first row, we're taking the 14th column,
第八排,我们选第八列 第一排,我们取第十四排

14:21 etc. And so log props at this plugs out all those log probabilities of the correct next character in
等,所以在此插插插点的日志道具将排除所有正确的下一个字符在

14:31 a sequence. So that's what that does and the shape of this or the size of it is of course 32,
序列。所以这就是它的作用,它的形状或大小 当然是32

14:37 because our batch size is 32. So these elements get plugged out and then their mean and the
因为我们的批量大小是32 所以这些元素被塞出 然后它们的平均和

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

14:44 negative of that becomes loss. So I always like to work with simpler examples to understand the
负的会变成损失。所以我总是喜欢用简单的例子来理解

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

14:51 numerical form of derivative. What's going on here is once we've plugged out these examples,
衍生物的数值形式。这里发生的事情是 一旦我们插入了这些例子,

derivative
导数 导数
反向传播与梯度

14:58 we're taking the mean and then the negative. So the loss basically, I can write it this way,
我们用的是平均值,然后是负值。所以损失基本上,我可以这样写,

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

15:04 is the negative of say a plus b plus c. And the mean of those three numbers would be say
代表一个加 b 加 c 的负数。 而这三个数字的平均值就是

15:09 negative, would divide three, that would be how we achieve the mean of three numbers a b c,
负,三分三,这就是我们如何达到bc3的平均值

15:14 although we actually have 32 numbers here. And so what is basically the loss by say like d a,
虽然我们这里有32个数字, 基本上损失的是什么?

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

15:21 right? Well, if we simplify this expression mathematically, this is negative one over
好吧,如果我们简化这个表达方式 数学学说,这是负的

expression
表达式 表达式
神经网络基础

15:26 three of a and negative plus negative one over three of b plus negative one over three of c.
3个和负+负1个和b3个以上,b+负1一个和c3个以上。

15:33 And so what is d loss by d a, it's just negative one over three. And so you can see that if we don't
所以,如果DA损失了多少, 它只是负1+3。所以你可以看到 如果我们不这样做

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

15:38 just have a b and c, but we have 32 numbers, then d loss by d, you know, every one of those
只有一个b和c,但我们有32个数字, 然后D损失,你知道,每一个

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

15:44 numbers is going to be one over n more generally, because n is the size of the batch 32 in this case.
数字将比一般数值为一至一 n以上,因为n是本案第32批的大小。

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

15:53 So d loss by d log props is negative one over n in all these places. Now, what about the other
在所有这些地方, dlog 道具的损耗是负的, N+N。 那么其他的呢?

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

16:03 elements inside log props? Because log props is large array, you see that log props are shaped
日对帐道的元素内原对帐道?由于日对帐口是大数阵列,所以您可以看到日对帐面的形状

16:08 as 32 by 27. But only 32 of them participate in the loss calculation. So what's the derivative of all
3乘32乘27,但其中只有32人参与损失计算。所以什么是所有

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

derivative
导数 导数
反向传播与梯度

16:15 the other most of the elements that do not get plucked out here? Well, their loss intuitively is
其它大部分元素不在这里被拔出来?

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

16:21 zero. Sorry, their gradient intuitively is zero. And that's because they did not participate in
零。对不起,他们的直觉梯度为零。这是因为他们没有参与

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

16:26 the loss. So most of these numbers inside this tensor does not feed into the loss. And so if we
损失。所以这个阵列中的大部分这些数字 都与损失无关,所以如果我们

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

16:32 were to change these numbers, then the loss doesn't change, which is the equivalent of we are
换了这些数字,然后损失没有变化, 也就是我们

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

16:37 saying that the derivative of the loss with respect to them is zero. They don't impact it.
说损失的衍生物是零,不会影响它

derivative
导数 导数
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

16:43 So here's a way to implement this derivative. Then we start out with torched at zeros of shape 32 by
有个方法可以执行这个衍生物。然后我们从32号形状零点开始

derivative
导数 导数
反向传播与梯度

16:49 27. Or let's just say, instead of doing this, because we don't want to hard code numbers,
或我们只是说, 而不是这样做, 因为我们不想硬代码号码,

16:54 let's do torched at zeros like log props. So basically, this is going to create an array
让我们在零点点燃火炬,像木日志道具。所以基本上,这将会创建一个阵列

16:59 of zeros exactly in the shape of log props. And then we need to set the derivative of
零的零正对对数道具形状。 然后我们需要设置

derivative
导数 导数
反向传播与梯度

17:05 negative one over n inside exactly these locations. So here's what we can do.
准确的这些位置内有1个负,我们只能这么做

17:10 The log props indexed in the identical way will be just set to negative one over zero,
以相同方式索引的日志道具将设置为负1+0

17:16 divide n, right, just like we derived here. So now let me erase all these reasoning.
分开 n,对,就像我们在这里的推理一样。现在让我抹去所有这些推理。

17:25 And then this is the candidate derivative for the log props. Let's uncomment the first line and
然后这是对数道具的候选衍生物。让我们取消第一行的内容,

derivative
导数 导数
反向传播与梯度

17:31 check that this is correct. Okay, so cmp ran. And let's go back to cmp. And you see that what it's
检查是否正确。 好了, 所以 cmp 运行了。 让我们回到cmp。 你看它是什么

17:40 doing is it's calculating if the calculated value by us, which is dt is exactly equal to t dot grad
是在计算我们计算得出的数值, 即 dt 是否等于 t dot grad

17:47 is calculated by pytorch. And then this is making sure that all the elements are exactly equal.
由燃烧器计算。然后,这确保了所有元素完全相等。

pytorch
PyTorch PyTorch
代码与工程

17:53 And then converting this to a single Boolean value, because we don't want to Boolean tensor,
然后把这个转换成一个单一的布尔值, 因为我们不想布林加强,

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

17:57 we just want to Boolean value. And then here we are making sure that, okay, if they're not exactly
我们只想让布林值。然后我们在这里确保,如果它们不是

18:03 equal, maybe they are approximately equal because of some floating point issues, but they're very,
平等,也许他们大致是平等的 因为一些浮点问题, 但是他们非常,

18:07 very close. So here we are using tors.allclose, which has a little bit of a wiggle available,
差不多了,所以我们用的是托盘, 电路,它有点摇晃,

18:13 because sometimes you can get very, very close. But if you use a slightly different
因为有时你非常接近,但如果你使用稍微不同

18:18 calculation, because of floating point arithmetic, you can get a slightly different result.
计算,由于浮动点算术,结果会略有不同。

18:23 So this is checking if you get an approximately close result. And then here we are checking the
这是在检查你是否得到近似接近的结果。然后我们在这里检查

18:28 maximum, basically the value that has the highest difference. And what is the difference and the
最大值,基本上就是有最大差值的值。

18:35 absolute value difference between those two. And so we are printing whether we have an exact
两者的绝对值差。 所以我们正在打印是否具有

18:39 equality and approximate equality. And what is the largest difference? And so here,
平等,和近似平等。什么是最大的差别?这里是这样的:

18:46 we see that we actually have exact equality. And so therefore, of course, we also have
因此,当然,我们也有

反向传播 · PyTorch 18:50

18:50 an approximate equality. And the maximum difference is exactly zero. So basically,
和最大差差是零 所以基本上

18:55 our D log props is exactly equal to what PyTorch calculated to be log props that grad in its
我们的日志道具完全等于 PyTorrch 计算为 原木道具的

pytorch
PyTorch PyTorch
代码与工程

19:02 back propagation. So so far, we're working pretty well. Okay, so let's now continue our back
继续后背,继续前背

back propagation
反向传播 反向传播
神经网络基础

19:07 propagation. We have that log props depends on props through a log. So all the elements of
传播。我们有日志道具 取决于通过日志的道具。所以所有的元素

19:13 props are being element wise applied log to. Now if we want deep props, then then remember
等离子体是智能应用日志的元素。 现在,如果我们想要深道具, 那么记住

19:20 your micrograph training, we have like a log node, it takes in props and creates log props.
你的显微镜训练,我们有像一个对数节点, 它吸收道具和创造日志道具。

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

19:26 And the props will be the local derivative of that individual operation log times the
道具将是该单操作的本地衍生物 将

derivative
导数 导数
反向传播与梯度

operation
运算 运算
神经网络基础

19:33 derivative loss with respect to its output, which in this case is D log props. So what is
其产出的衍生衍生物损失,在本案中,就是Dlog 道具。

derivative
导数 导数
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

19:37 the local derivative of this operation? Well, we are taking log element wise. And we can come
本地衍生物?

derivative
导数 导数
反向传播与梯度

operation
运算 运算
神经网络基础

19:42 here and we can see well from all flies your friend that D by DX of log of X is just simply one of our
这儿我们可以看到,从所有苍蝇的你朋友身上 我们可以看到,DDDXXXX X的XX对X日志 只是我们的一个

19:47 X. So therefore, in this case, X is props. So we have D by DX is one over X, which is one of our
因此,在本案中,X就是道具。所以我们用DX来DX是X的一对一, 也就是我们的一个

19:55 props. And then this is the local derivative. And then times we want to chain it. So this is chain
等主控部件。 然后这是本地衍生物, 然后我们想把它链链起来。 所以这就是链

derivative
导数 导数
反向传播与梯度

20:00 rule times D log props. And let me comment this and let me run the cell in place. And we see
规则值乘以 D 日对帐面,让我评论这个,然后我运行细胞。我们可以看到

20:07 that the derivative of props as we calculated here is exactly correct. And so notice here how this works
和我们在这里计算到的道具的衍生物 完全正确。所以请注意这里如何使用

derivative
导数 导数
反向传播与梯度

20:15 props that are props is going to be inverted and then element wise multiplied here. So if your
支持的反向, 将被倒转, 然后元素智慧乘法乘以此处。 所以如果您

20:21 props is very, very close to one, that means your network is currently predicting the character
支持的 psps is very, very clos atly to one, 这意味着您的网络正在预测字符

20:26 correctly, then this will become one over one and the log props is just gets passed through.
正确,然后它会变成一个 一个反一 和日志道具只是通过。

20:31 But if your probabilities are incorrectly assigned, so if the correct character here
但如果您的可能性分配错误, 如果正确的话

20:36 is getting a very low probability, then 1.0 dividing by it will boost this
概率非常低, 1.0 平分一成就会提升这个

probability
概率 概率
数学与微积分

20:43 and then multiply by the other props. So basically what this line is doing intuitively
并乘以其他道具。所以基本上,这条线是凭直觉做哪些事情

20:47 is is taking the examples that have a very low probability currently assigned,
举目前所分配的概率非常低的例子,

probability
概率 概率
数学与微积分

20:51 and it's boosting their gradient. You can look at it that way. Next up is count some
并且加速了他们的梯度。 你可以这样看待它。接下来是数数一些

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

20:57 imp. So we want the river of this. Now let me just pause here and kind of introduce what's
imm. 所以我们想要这条河的河流。 现在让我在这里暂停一下,介绍一下

21:05 happening here in general, because I know it's a little bit confusing. We have the logists that
通常这里发生,因为我知道这有点混乱

21:09 come out of the neural net. Here, what I'm doing is I'm finding the maximum in each row,
这儿,我要做的是找到每一行的最高值

neural net
神经网络 神经网络
神经网络基础

21:14 and I'm subtracting it for the purpose of numerical stability. And we talked about how,
为了数字稳定起见,我减了它。

21:18 if you do not do this, you run numerical issues if some of the logists take on two
如果您不这样做, 如果有些日志用户在两个日期上运行数字问题

21:22 large values because we end up exponentiating them. So this is done just for safety, numerically.
宏值,因为我们最终会把它们推高。所以这只是为了安全起见,数字上来说。

21:29 Then here's the exponentiation of all the logists to create our counts, and then we want to take
然后我们想拿下

21:36 the sum of these counts and normalize so that all of the props sum to 1. Now here instead of using
和这些计数的总和,并实现正常化, 所有道具和道具总和为1. 现在在这里,而不是使用

21:42 one over count sum, I use raised to the power of negative 1. Mathematically, they are identical.
以负1的功率升起,从数学上看,它们完全相同。

21:48 I just found that there's something wrong with the PyTorch implementation of the backward
发现皮托尔克的后方 实施反向的

pytorch
PyTorch PyTorch
代码与工程

implementation
实现 实现
代码与工程

21:51 passive division, and it gives like a weird result, but that doesn't happen for a star star negative
恒星负负的,不会发生

21:57 one. So I'm using this formula instead. But basically, all that's happening here is we got the logists,
一种。所以我用这个公式代替这个公式, 但基本上,这里发生的一切就是我们得到了日志员,

22:03 we want to exponentiate all of them, and we want to normalize the counts to create our
我们希望将所有他们都推上他们的前列,我们希望使罪魁祸首正常化,从而建立我们的

22:07 probabilities. It's just that it's happening across multiple lines. So now here we want to
概率概率。只是它正在发生, 它在多条线上。所以现在我们想在这里

22:18 first take the derivative, we want to back propagate into counts sum if and then into counts as well.
首先取出衍生物, 我们想回到传播 以计数总和,如果和随后 也计数。

derivative
导数 导数
反向传播与梯度

22:24 So what should be the counts sum if now we actually have to be careful here because
那么,如果我们现在真的需要小心,那么,我们到底应该算什么?

22:30 we have to scrutinize and be careful with the shapes. So counts that shape and then counts
我们必须仔细检查和小心形状。 所以计数形状然后计数

22:36 some in that shape are different. So in particular counts is 32 by 27, but this counts some in
某些形状的有些形状不同。因此,具体计数是32乘27,但其中有一些在

22:44 is 32 by one. And so in this multiplication here, we also have an implicit broadcasting
乘以32,一个是32。所以在这个乘法中, 我们也有暗含的广播

broadcasting
广播 广播
数学与微积分

22:50 that PyTorch will do because it needs to take this column tensor of 32 numbers and replicate it
平托尔奇会这样做,因为它需要使用32个数字的柱状元并复制它

pytorch
PyTorch PyTorch
代码与工程

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

22:55 horizontally 27 times to align these two tensors so it can do an element twice multiply. So really
水平对齐这两个 10 组, 使元素乘以2倍。 所以真的

23:01 what this looks like is the following using a toy example again. What we really have here is just
是这样的,下面又用一个玩具的例子。我们这里的真面目就是

23:07 props is counts times consum if so it's a C equals a times B, but a is three by three and B is just
质量是计算乘数的计算乘以计算,如果是 C 等于乘以 B,但a 等于 三乘三,B 只是

23:14 three by one a column tensor. And so PyTorch internally replicated this elements of B and it did that
皮托尔切内部复制了B元素,它做了

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

pytorch
PyTorch PyTorch
代码与工程

23:21 across all the columns. So for example, B one, which is the first element of B would be replicated
横跨所有列。例如,B 1,B的第一个元素,B1将复制

23:26 here across all the columns in this multiplication. And now we're trying to back
乘数中所有列的纵横交叉

23:30 propagate through this operation to count some in. So when we were calculating this
通过此操作传播到计算部分。 所以当我们计算时

operation
运算 运算
神经网络基础

23:35 derivative, it's important to realize that these two, this looks like a single operation,
派生的,重要的是要认识到 这两个,这看起来像一个操作

derivative
导数 导数
反向传播与梯度

operation
运算 运算
神经网络基础

23:41 but actually is two operations applied sequentially. The first operation that PyTorch did is it took
皮托尔奇的第一个操作是它采取了

operation
运算 运算
神经网络基础

pytorch
PyTorch PyTorch
代码与工程

23:47 this column tensor and replicated it across all the across all the columns basically 27 times.
基本上在所有列中复制27次。

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

23:54 So that's the first operation, it's a replication. And then the second operation is the
这是第一个,一个复制。第二个是

operation
运算 运算
神经网络基础

23:58 multiplication. So let's first back prop through the multiplication. If these two arrays were
乘法。让我们先从乘法中返回支持。如果这两个阵列是

24:04 of the same size and we just have A and B, both of them three by three, then how do we back propagate
大小相同,我们只有A和B, 两人三三三, 然后我们怎么回播

24:11 through a multiplication? So if we just have scalars and not tensors, then if you have C equals A
乘以乘法的乘数。所以如果我们只是有电弧而不是数以万计,那么如果你有C等于A

24:16 times B, then what is the derivative of C with respect to B? Well, it's just A. And so that's
B乘以B,那么C的衍生物相对于B是什么?

derivative
导数 导数
反向传播与梯度

24:22 the local derivative. So here in our case, undoing the multiplication and backprop again
本地衍生物。所以在这里,我们的情况是, 取消乘法和反向反向

derivative
导数 导数
反向传播与梯度

backprop
反向传播 反向传播
反向传播与梯度

24:28 through just the multiplication itself, which is element wise, is going to be the local
仅仅通过乘以本身, 也就是明智的因素, 将是本地的

24:33 derivative, which in this case is simply counts, because counts is the A. So it's the local derivative
衍生物,在这个案例中,它只是计算, 因为计数是 A。 所以它是本地的衍生物

derivative
导数 导数
反向传播与梯度

24:41 and then times because the chain rule D props. So this here is the derivative or the gradient,
并且有时是因为链条规则D道具。所以这里是衍生物或梯度,

chain rule
链式法则 链式法则
反向传播与梯度

derivative
导数 导数
反向传播与梯度

24:49 but with respect to replicated B. But we don't have a replicated B, we just have a single B column.
但是关于复制B,我们没有复制B, 我们只有一个B列。

24:55 So how do we now back propagate through the replication? And intuitively, this B one
那么我们现在如何通过复制来回播?直觉上,这个B

25:01 is the same variable and it's just reused multiple times. And so you can look at it as being equivalent
是一个相同的变量,它只是重复使用过多次。所以你可以把它看作等量

25:08 to a case with encountered in micrograd. And so here I'm just pulling out a random graph we used
在一个微型研究生遇到的案例中。所以我在这里 正在绘制一个随机图表,我们用

micrograd
micrograd 自动微分库 micrograd 自动微分库
神经网络基础

25:13 in micrograd. We had an example where a single node has its output feeding into two branches of
我们有一个例子,一个单一节点的输出 被注入两个分支

micrograd
micrograd 自动微分库 micrograd 自动微分库
神经网络基础

25:21 basically the graph until the last function. And we're talking about how the correct thing to
图形基本上直到最后函数。我们谈论的是

25:25 do in the backward pass is we need to sum all the gradients that arrive at any one node. So
后空口是我们需要将到达任何一个节点的所有梯度相加。所以,

backward pass
反向传播 反向传播
神经网络基础

25:31 across these different branches, the gradients would sum. So if a node is used multiple times,
在这些不同的分支之间,梯度会和。所以如果节点被多次使用,

25:36 the gradients for all of its uses sum during back propagation. So here B one is used multiple
后传传播期间所有使用其使用和的渐变。 因此这里 B 1 使用的是多个

back propagation
反向传播 反向传播
神经网络基础

25:43 times in all of these columns. And therefore the right thing to do here is to sum horizontally
在所有这些列中的时间值。 因此,正确的做法是横向和

25:49 across all the rows. So we'll just sum in dimension one. But we want to retain this
横行各行。所以我们只要在一维中相加, 但我们想保留这个

dimension
维度 维度
数学与微积分

25:55 dimension so that the so that count some and and its gradient are going to be exactly the same
使计算某些和梯度的大小与

dimension
维度 维度
数学与微积分

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

26:01 shape. So we want to make sure that we keep them as true. So we don't lose this dimension.
形状。 所以我们要确保我们保持它们的真实性。 这样我们就不会失去这个维度。

dimension
维度 维度
数学与微积分

26:06 And this will make the count some and be exactly shape 32 by one. So revealing this
这会让数数数成部分, 并且精确形状32 乘一个。 所以要显示这个

26:13 comparison as well and running this, we see that we get an exact match. So this derivative is
比较和比较,运行这个,我们看到我们得到一个精确匹配。所以这个衍生物是

derivative
导数 导数
反向传播与梯度

26:19 exactly correct. And let me erase this. Now let's also back propagate into counts,
完全正确,让我来抹掉这个 现在让我们也重新传播成数

26:27 which is the other variable here to create props. So from props to count some and we just did that,
也就是另一个变量在这里创建道具。所以从道具到计数,我们刚刚做了

26:33 let's go into counts as well. So the counts will be the counts are a so DC by DA is just B.
计数结果就是...

26:44 So therefore it's count some and then times chain rule the props. Now consum in is 32 by one,
所以,它会计算一些 然后链条决定道具。现在的消费是32乘1

chain rule
链式法则 链式法则
反向传播与梯度

26:54 deep props is 32 by 27. So those will broadcast fine and will give us the counts.
深道是32乘27,所以这些可以播放好 而且会告诉我们的数

27:03 There's no additional summation required here. There will be a broadcasting that happens in
这里不需要额外的加和。

broadcasting
广播 广播
数学与微积分

27:09 this multiply here because consum in needs to be replicated again to correctly multiply
此处此乘数在此, 因为消费需要复制才能正确增加

27:14 deep props. But that's going to give the correct result. So as far as the single operation is
深道具,但结果会正确

operation
运算 运算
神经网络基础

27:20 concerned, so we backprop it from props to counts, but we can't actually check the derivative of
有关,所以我们把它从道道道 反向计数, 但我们不能实际上检查

backprop
反向传播 反向传播
反向传播与梯度

derivative
导数 导数
反向传播与梯度

27:27 counts. I have it much later on. And the reason for that is because count some in depends on
。我很久以后才有它,原因在于

27:33 counts. And so there's a second branch here that we have to finish, because consum in backprop
。所以这里还有一个分支,我们必须完成它, 因为在后方的消费

backprop
反向传播 反向传播
反向传播与梯度

27:38 against into counts some and count some will backprop into counts. And so counts is a node that
计数,计数,算数,算算,算算。计数,是一个节点,

backprop
反向传播 反向传播
反向传播与梯度

27:44 is being used twice. It's used right here into props and it goes through this other branch through
被使用过两次。它在这里被用于道具, 通过这个分支通过

27:48 count some in. So even though we've calculated the first contribution of it, we still have to
计算了其中的首笔贡献,我们仍必须

27:53 calculate the second contribution of it later. Okay, so we're continuing with this branch.
稍后计算第二笔捐款。 好的,所以我们继续使用这个分支。

27:59 We have the derivative for counts of M now we want the derivative counts some.
我们有M的衍生物 来计算M的计数 现在我们要的是衍生物的计数

derivative
导数 导数
反向传播与梯度

28:03 So the counts some equals what is the local derivative of this operation. So this is
所以计算数数中有些数等于什么是 此次操作的本地衍生物。 因此,这是

derivative
导数 导数
反向传播与梯度

operation
运算 运算
神经网络基础

28:08 basically an element wise one over counts some. So counts some raised to the power of negative one
基本上是一个智者元素, 而不是一些。 所以有些元素被提升到负的元素

28:14 is the same as one over counts some. If we go to all from alpha, we see that x to the negative one
与数数数的一对一相同。如果我们从阿尔法到所有人,我们可以看到x到负

28:19 d by d by d x of it is basically negative x to the negative two, right one negative one over
d d 乘 d x 等于 负 2,右1 等于

28:25 a squared is the same as negative x to the negative two. So the counts some here will be
a 平方等于负 x 负 2 的负 x。 因此这里的一些计数将是

28:32 local derivative is going to be negative counts some to the negative two. That's the local derivative
本地衍生物是负数, 部分为负数。 这是本地衍生物

derivative
导数 导数
反向传播与梯度

28:40 times chain rule, which is d counts some in. So that's d counts some. Let's uncomment this and
时间 规则, 也就是 d 值一些。 所以d 值 数一些。 让我们不提这一点和问题

chain rule
链式法则 链式法则
反向传播与梯度

28:50 check that I am correct. Okay, so we have perfect equality. And there's no sketching
好吧,我们完全平等,没有草图

28:57 us going on here with any shapes because these are of the same shape. Okay, next up we want to
因为我们的形状是相同的。

29:02 back propagate through this line. We have that counts some is counts that some along the rows.
反向通过这条线传播。我们有这样的数字,有些是沿着各行的数值。

29:07 So I wrote out some help here. We have to keep in mind that counts of course is 32 by 27 and
因此我写了些帮助。我们必须记住,当然, 数数是32乘27,

29:14 counts some is 32 by one. So in this back propagation, we need to take this column of derivatives
数数为32乘1。所以在这种后向传播中, 我们需要使用这个衍生物列

back propagation
反向传播 反向传播
神经网络基础

29:21 and transform it into a array of derivatives, two dimensional array. So what is this operation
并转换成一系列衍生物,两个维数组。这个操作是什么?

operation
运算 运算
神经网络基础

29:28 doing? We're taking some kind of an input like say a three bytes matrix A, and we are summing up the
我们用某种输入方式 比如说三个字字字矩阵A

matrix
矩阵 矩阵
数学与微积分

29:33 rows into a column tensor B, B1, B2, B3. That is basically this. So now we have the derivatives
列的 ARor B, B1, B2, B3。 基本上就是这个。 现在我们有了衍生物

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

29:41 of the loss with respect to B, all the elements of B. And now we want to derivative loss with
B损失的损失,B的所有元素。 现在,我们想将

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

derivative
导数 导数
反向传播与梯度

29:46 respect to all these little a's. So how do the B's depend on the a's is basically what
尊重所有这些小A的。那么B是如何依赖A的呢?

29:52 we're after. What is the local derivative of this operation? Well, we can see here that B1
我们在找,这次行动的地方衍生物是什么?

derivative
导数 导数
反向传播与梯度

operation
运算 运算
神经网络基础

29:57 only depends on these elements here. The derivative of B1 with respect to all of these
仅取决于这些元素。 B1的衍生物,与所有这些元素相关的

derivative
导数 导数
反向传播与梯度

30:03 elements down here is zero. But for these elements here like a1, a1, a1, 2, etc. The local derivative
下面的元素为零。 但是对于这些元素, 如 a1, a1, A1, a1、2 等。 本地衍生品

derivative
导数 导数
反向传播与梯度

30:09 is one, right? So dB1 by dA11, for example, is one. So it's one, one and one. So when we have
是一个,对不对?例如,DB1 by dA11,就是一个。所以它是一个,一个和一个。当我们有了

30:18 the derivative of loss with respect to B1, the local derivative of B1 with respect to these
B1的当地B1的衍生物,即B1的当地衍生物,与这些损失有关的衍生物

derivative
导数 导数
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

30:24 inputs is zeroes here, but it's one on these guys. So in the chain rule, we have the local derivative
输入是零,但就是一对一。所以在链规则中,我们有本地衍生物

chain rule
链式法则 链式法则
反向传播与梯度

derivative
导数 导数
反向传播与梯度

30:31 times sort of the derivative of B1. And so because the local derivative is one on these three elements,
B1的衍生物的乘数, 因为本地衍生物是这三个元素中的一个

derivative
导数 导数
反向传播与梯度

30:39 the local derivative multiplying the derivative of B1 will just be the derivative of B1.
B1的衍生物只是B1的衍生品。

derivative
导数 导数
反向传播与梯度

30:45 And so you can look at it as a router. Basically an addition is a router of gradient.
这样你就可以把它看成一个路由器。基本上,添加一个路由符就是梯度的路由器 。

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

30:50 Whatever gradient comes from above, it just gets routed equally to all the elements that participate
无论梯度来自上面, 它都平等地被所有参与元素所利用

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

30:54 in that addition. So in this case, the derivative of B1 will just flow equally to the derivative of
因此在这种情况下,B1的衍生物将同样流向

derivative
导数 导数
反向传播与梯度

31:00 a11, a12, and a13. So if we have a derivative of all the elements of B, and in this column tensor,
a11, a12, 和 a13。 所以如果我们有B的所有元素的衍生物, 在这个柱中,

derivative
导数 导数
反向传播与梯度

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

31:07 which is d counts sum that we've calculated just now, we basically see that what that amounts to
也就是我们刚刚计算出来的 d 计数和, 我们基本上看到

31:14 is all of these are now flowing to all these elements of a, and they're doing that horizontally.
这些都是现在流入所有元素的A, 它们横向进行。

31:20 So basically what we want is we want to take the d counts sum of size 30 by 1, and we just want to
所以我们想要的,基本上就是 我们想把30乘1的D数总和 取下30乘1号的D数,我们只是想

31:26 replicate it 27 times horizontally to create 32 by 27 array. So there's many ways to implement
横向复制27倍,以创建 32 乘 27 矩阵。所以有很多方法可以执行

31:33 this operation. You could of course just replicate the tensor. But I think maybe one clean one is
当然你可以复制电压。 但我想,也许一个干净的

operation
运算 运算
神经网络基础

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

31:38 that d counts is simply torsion dot once like, so just an two dimensional arrays of once in the shape
d 计数是简单的 撕裂点点,曾经一样, 所以只要一个两个维数数 在一个形状中一次

31:47 of counts, so 32 by 27 times d counts sum. So this way we're letting the broadcasting here
以32乘27 d 计时总和。所以我们让广播在这里

broadcasting
广播 广播
数学与微积分

31:56 basically implement the replication. You can look at it that way. But then we have to also be
基本上执行复制。 您可以这样看待它。 但是,我们也必须

32:00 careful because d counts was already calculated. We calculated it earlier here. And that was
注意,因为 d 计数已经计算。 我们在这里早些时候计算了它。

32:07 just the first branch. And we're now finishing the second branch. So we need to make sure that
并且我们现在要完成第二分支 所以我们需要确保

32:11 these gradients add so plus equals. And then here, let's comment out the comparison. And let's make
这些梯度加起来的加起来等于等值。然后在这里,让我们来评论比较。我们做

32:20 sure crossing fingers that we have the correct result. So pytorch agrees with us on this gradient
确定我们有正确的结果。 因此,Pytorch同意我们关于这个梯度的意见

pytorch
PyTorch PyTorch
代码与工程

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

32:26 as well. Okay, hopefully we're getting a hang of this now counts as an element y x of norm logits.
希望我们掌握了这个信息 算作正常日志的yx元素

norm
范数 范数
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

32:33 So now we want the normal logits. And because it's an element of operation, everything is very
现在我们想要正常的登录记录, 因为它是操作的一个元素, 一切都非常

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

operation
运算 运算
神经网络基础

32:38 simple. What is the local derivative of e to the x? It's famously just e to the x. So this is the
e 到 x 的本地衍生物是什么?

derivative
导数 导数
反向传播与梯度

32:45 local derivative. That is the local derivative. Now we already calculated it and it's inside counts.
本地衍生物,这是本地衍生物 现在我们已经计算过了,内部计数

derivative
导数 导数
反向传播与梯度

32:52 So we made as well potentially just reuse counts. That is the local derivative times
所以我们做了很多可能只是重新利用计算。那是本地衍生时代

derivative
导数 导数
反向传播与梯度

32:57 d counts. Funny as that looks counts time d counts is the derivative on the normal logits.

d. 有趣的是,看起来算时间的计算是正常登录上的衍生物。

derivative

导数 导数

反向传播与梯度

logits

对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax

神经网络基础

33:06 And now let's erase this and let's verify and it looks good. So that's normal logits.

现在让我们去掉这个,我们核查一下 看起来不错,这是正常的登录记录

logits

对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax

神经网络基础

33:14 Okay, so we are here on this line now, the normal logits. We have that and we're trying to

好了,我们现在在这条线上, 正常的登录记录。我们有这个,我们正试图

logits

对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax

神经网络基础

33:20 calculate the logits and the logit maxes. So back propagating through this line.

计算日志和日志最大值。 所以返回此行后进行传播 。

logits

对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax

神经网络基础

33:25 Now we have to be careful here because the shapes again are not the same. And so there's

现在我们必须小心,因为形状又不一样了。所以有

33:29 an implicit broadcasting happening here. So normal logits has this shape 32 by 27 logits
暗含的广播在这里进行。 所以普通登录有这个形状 32 乘 27 个登录

broadcasting
广播 广播
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

33:35 does as well. But logit maxes is only 32 by 1. So there's a broadcasting here in the minus.
最多为32x1,所以这里有一个广播在负数中

broadcasting
广播 广播
数学与微积分

33:42 Now here I tried to sort of write out a toy example again. We basically have that this is our c equals
现在我想写一个玩具的例子。我们基本上都认为这是我们的C等

33:49 a minus b. And we see that because of the shape, these are three by three, but this one is just a
a 减 b。我们发现,由于形状的形状, 这些是三乘三, 但这个只是

33:54 column. And so for example, every element of c, we have to look at how it came to be. And every
例如,例如,c的每个元素, 每一个元素, 我们不得不看看它是如何来的。

34:00 element of c is just the corresponding element of a minus basically that associated b. So it's
c 元素只是一个减的对应元素,基本上就是相关 b. 所以它

34:09 very clear now that the derivatives of every one of these C's with respect to their inputs are one
现在很清楚了,每一个C的衍生物 就其投入而言,都是

34:16 for the corresponding a and it's a negative one for the corresponding b. And so therefore,
对应的a和对应的b是负的,所以

34:23 the derivatives on the C will flow equally to the corresponding a's and then also to the
C的衍生物将平等流向相应的a

34:30 corresponding b's. But then in addition to that, the b's are broadcast. So we'll have to do the
之后B's被广播了

34:35 additional sum just like we did before. And of course, derivatives for b's will undergo a minus
当然,B的衍生物将经历一个负负

34:41 because the local derivative here is negative one. So dc 32 by db3 is negative one.
因为本地衍生物是负的。 所以dc 32 乘以 db3 是负的 。

derivative
导数 导数
反向传播与梯度

34:49 So let's just implement that. Basically, d logits will be exactly copying the derivative on norm
所以让我们执行这个。基本上,D登录将完全复制衍生物

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

derivative
导数 导数
反向传播与梯度

34:55 logits. So d logits equals d norm logits, and I'll do a dot clone for safety. So we're just
登录。 所以, d 登录等于 d 规范登录, 我会做一个点克隆, 安全性。 所以我们只是

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

norm
范数 范数
数学与微积分

35:03 making a copy. And then we have that d logit maxis will be the negative of d norm logits
然后我们有了D 日志最大值将是 d 规范登录的负值

norm
范数 范数
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

35:12 because of the negative sign. And then we have to be careful because logit maxis is a column.
因为负符号。 然后我们必须小心,因为登录最大值是一列。

35:20 And so just like we saw before, because we keep replicating the same elements across all the
因为我们不断在

35:27 columns, then in the backward pass, because we keep reusing this, these are all just like
列,然后在后过, 因为我们不断重复使用这个, 这些都就像

backward pass
反向传播 反向传播
神经网络基础

35:33 separate branches of use of that one variable. And so therefore, we have to do a sum along one
使用该变量的分分支。 因此, 我们必须在一子一个变量上做一个总和

35:39 would keep them equals true, so that we don't destroy this dimension. And then d logit maxis
保持它们等同真实, 这样我们就不会摧毁这个维度。 然后登入最大值

dimension
维度 维度
数学与微积分

35:44 will be the same shape. Now we have to be careful because this d logits is not the final d logits.
将会是相同的形状。 现在我们必须小心, 因为这个 d 登录不是最后的 d 登录 。

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

35:50 And that's because not only do we get gradient signal into logits through here,
这是因为我们不仅把梯度信号 输入到登录中

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

35:55 but logit maxis is a function of logits. And that's a second branch into logits. So this is
登录时, 登录最大值是登录的函数。 这是在登录中第二个分支。 所以这是

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

36:00 not yet our final derivative for logits, we will come back later for the second branch.
还没有最后的衍生物来登入 我们稍后会回来买第二支分支

derivative
导数 导数
反向传播与梯度

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

36:05 For now, the logit maxis is the final derivative. So let me uncomment this cmp here and let's just run
现在,对日最大值是最终的衍生物。让我在这里解开这个小混凝土,让我们运行

derivative
导数 导数
反向传播与梯度

36:11 this. And logit maxis if by torch agrees with us. So that was the derivative into through this line.
并登录最大限度, 如果火炬同意我们的话。 所以这是通过这条线的衍生物。

derivative
导数 导数
反向传播与梯度

36:21 Now before we move on, I want to pause here briefly and I want to look at these logit maxis
现在在我们继续之前,我想在这里简短地暂停一下 我想看看这些登录最大值

36:25 and especially their gradients. We've talked previously in the previous lecture
特别是他们的梯度。我们以前在上个讲座上谈过了

36:29 that the only reason we're doing this is for the numerical stability of the soft max that
我们之所以这么做 是因为软体的数值稳定性

36:33 we are implementing here. And we talked about how if you take these logits for any one of these
我们在这里执行。我们讨论过,如果你把这些登录 用于任何这些

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

36:38 examples, so one row of this logits tensor, if you add or subtract any value equally to all the elements,
示例,所以如果所有元素的数值均相等地增加或减去任何值,则本对数单的一行"拉度"

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

36:46 then the value of the probes will be unchanged. You're not changing the soft max. The only
探测器的价值不会改变。

36:51 thing that this is doing is it's making sure that x doesn't overflow. And the reason we're
这样做的,就是确保x不会溢出。我们之所以

36:55 using a max is because then we are guaranteed that each row of logits, the highest number,
使用最大值是因为这样我们就能保证 每排记录数量最高

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

37:00 is zero. And so this will be safe. And so basically what that has were precautions.
零是零。所以这样就安全了。基本上,这就是预防措施。

37:09 If it is the case that changing logit maxis does not change the probes and therefore does not
如果改变日志最大值不会改变探测器,因此不会改变探测器

37:14 change the loss, then the gradient on logit maxis should be zero, right? Because saying those
更改损失, 然后对日对数的梯度应该为零, 对不对? 因为说这些

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

37:19 two things is the same. So indeed we hope that this is very, very small numbers. Indeed we hope
两件事是相同的。我们的确希望这个数目是很少的,我们的确希望

37:24 this is zero. Now because of floating point sort of wonkiness, this doesn't come out exactly zero,
零点,因为浮点的温奇尼

37:31 only in some of the rows it does. But we get extremely small values like 1e negative 9 or 10.
只在一些行中它能做到。但我们得到的值极小,比如 1 e 负 9 或 10 。

37:36 And so this is telling us that the values of logit maxis are not impacting the loss as they
因此,这告诉我们 日志最大值不会影响损失

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

37:41 shouldn't. It feels kind of weird to back propagate through this branch honestly, because
不应该,因为要诚实地通过这个分支 进行反向宣传感觉很奇怪

37:48 if you have any implementation of like f dot cross entropy and pytorch and you
如果你有像fdot 交叉 entropy和Pytorch这样的执行,还有你

implementation
实现 实现
代码与工程

cross entropy
交叉熵 交叉熵
数学与微积分

37:52 block together all of these elements and you're not doing the back propagation piece by piece,
把这些元素都堵在一起,你不会一一做后台传播片

back propagation
反向传播 反向传播
神经网络基础

37:56 then you would probably assume that the derivative through here is exactly zero.
然后你可能假设这里的衍生物 完全为零

derivative
导数 导数
反向传播与梯度

38:01 So you would be sort of skipping this branch because it's only done for numerical stability.
所以你会跳过这个分支 因为它只是为了数字稳定

38:09 But it's interesting to see that even if you break up everything into the full atoms and you
但有趣的是,即使你把一切 都分解成原子

38:13 still do the computation as you'd like with respect to numerical stability, the correct thing
仍然按照您想要的数值稳定性进行计算,正确的事情

38:17 happens and you still get a very, very small gradients here basically reflecting the fact that
发生时,你仍然有一个非常, 非常小的梯度 在这里基本上反映的事实

PyTorch · 向量 38:23

38:23 the values of these do not matter with respect to the final loss. Okay so let's now continue
与最后损失无关。 好吧,让我们继续

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

38:28 back propagation through this line here. We've just calculated the logit maxis and now we want
我们刚刚计算了对数最大值,现在我们想要

back propagation
反向传播 反向传播
神经网络基础

38:33 to back prop into logits through this second branch. Now here of course we took logits and
返回到通过第二分支的登录中。现在在这里,我们当然采取了登入和

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

38:38 we took the max along all the rows and then we looked at its values here. Now the way this
我们沿着所有行的排排 取了最大数 然后我们在这里看了它的价值观。现在,这样

38:43 works is that in pytorch, this thing here, the max returns both the values and it returns
工作是,在烧火机中,这个东西在这里,最大返回值,然后返回

pytorch
PyTorch PyTorch
代码与工程

38:52 the indices at which those values to column the maximum value. Now in the forward pass we only
将这些值乘以列中的最大值的指数。现在在向前传动时,我们仅

forward pass
前向传播 前向传播
神经网络基础

38:57 used values because that's all we needed but in the backward pass it's extremely useful to know
使用值是因为这是我们需要的 但是在后过道上 了解这些非常有用

backward pass
反向传播 反向传播
神经网络基础

39:02 about where those maximum values occurred and we have the indices at which they occurred
最高值发生地点和我们拥有的指数

39:07 and this will of course help us do the back propagation because what should the backward
这当然会帮助我们做后传播 因为落后者应该做什么

back propagation
反向传播 反向传播
神经网络基础

39:12 pass be here in this case? We have the logit tensor which is 32 by 27 and in each row we find the maximum
此时这里有单点阵列, 时速32乘27, 每行最多能找到

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

39:18 value and then that value gets plucked out into logit maxis and so intuitively basically the
然后这个值被提取出来 进入对数最大值,所以直觉地基本上

39:26 derivative flowing through here then should be 1 times the local derivatives is 1 for the
本地衍生物的1倍于

derivative
导数 导数
反向传播与梯度

39:33 appropriate entry that was plucked out and then times the global derivative of the logit maxis.
适当的条目被提取出来,然后乘以对数最大值的全球衍生物。

derivative
导数 导数
反向传播与梯度

39:40 So really what we're doing here if you think through it is we need to take the delogit maxis
所以如果你仔细思考 我们在这里要做的是 我们需要采取 脱落最大量

39:45 and we need to scatter it to the correct positions in these logits from where the maximum values came
我们需要分散到这些日志中正确的位置, 并且我们需要从最大值的来处

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

39:53 and so I came up with one line of code sort of that does that let me just erase a bunch of stuff
所以我想出了一条代码线 排序的代码排序 做那让我只是擦去一堆东西

39:59 here. So the line of you could do it kind of very similar to what we done here where we create
所以你们可以在这里做一些类似的事情 和我们在这里做的非常相似的事, 我们创造

40:04 a zeros and then we populate the correct elements so we use the indices here and we would set them to
零为零,然后我们弹出正确的元素,这样我们就在这里使用指数,然后设定它们

40:11 be 1 but you can also use one hat so f dot one hat and then I'm taking the logit that max over the
仅1岁,但你也可以用一顶帽子 所以一顶帽子就点一个 然后我要登入

40:20 first dimension that indices and I'm telling PyTorch that the dimension of every one of these
指数的一维一维,我告诉皮托尔奇 这些指数中的每一个

dimension
维度 维度
数学与微积分

pytorch
PyTorch PyTorch
代码与工程

40:27 tensors should be 27 and so what this is going to do is okay I apologize this is crazy,
发光器应该27岁 所以这能做的是... ...没关系,我道歉 这是疯狂的,

40:36 PyTorch .i am sure of this it's really just an array of where the maxis came from in each row
皮托尔奇.I肯定这确实只是一列最大值的阵列

pytorch
PyTorch PyTorch
代码与工程

40:43 and that element is 1 and the all the other elements are 0 so it's a one-hot vector in each row
元素为 1, 而所有其他元素为 0, 所以它是每行的一热矢量

vector
向量 向量
数学与微积分

40:49 and these indices are now populating a single 1 in the proper place and then what I'm doing
这些指数现在正在 适当的地方 以一个1的指数来计算 然后我要做什么

40:55 here is I'm multiplying by the logit maxis and keep in mind that this is a column of 32 by 1
记住这是32乘1的列

41:03 and so when I'm doing this times the logit maxis the logit maxis will broadcast and that column will
当我做这个时, 日对账最大值是 日对帐最大值将广播

41:10 you know get replicated and then element wise multiply will ensure that each of these just
和智者乘数元素 将确保这些

41:15 gets routed to whichever one of these bits is turned on and so that's another way to implement
被选择到这些位子中 哪个被打开的路径, 所以这是另一种执行方式

41:21 this kind of an operation and both of these can be used I just thought I would show an equivalent
这两种行动都可以使用 我只是想展示一个相当的

operation
运算 运算
神经网络基础

41:28 way to do it and I'm using plus equals because we already calculated the logits here and this is
这样做的方式,我使用加加等于等于 因为我们已经计算了这里的登录

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

41:33 now the second branch so let's look at logits and make sure that this is correct and we see
第二分支现在第二分支 所以让我们看看登入 并确保这是对的,我们可以看到

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

41:41 that we have exactly the correct answer. Next up we want to continue with logits here that is
答案是正确的。接下来我们要继续在这里登录

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

41:47 an outcome of a matrix multiplication and a bias offset in this linear layer so I've printed out the
矩阵乘法的结果 和线性层的偏偏偏偏移抵消 所以我打印了

matrix
矩阵 矩阵
数学与微积分

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

41:55 shapes of all these intermediate tensors we see that logits is of course 32 by 27 as we've just seen
我们看到的这些中间的抗拉形状 登入当然是32乘27

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

42:02 then the h here is 32 by 64 so these are 64 dimensional hidden states and then this w matrix
然后是32x64,然后是34维隐藏状态 然后是这个矩阵

matrix
矩阵 矩阵
数学与微积分

42:09 projects those 64 dimensional vectors into 27 dimensions and then there's a 27 dimensional
将64维矢量投射到27维 然后是27维

42:15 offset which is a one dimensional vector now we should note that this plus here actually broadcasts
中此值是一维矢量, 现在我们应注意,

vector
向量 向量
数学与微积分

42:21 because h multiplied by w2 will give us a 32 by 27 and so then this plus b2 is a 27 dimensional
h 因为h乘以 w2 将给我们32乘27 所以这个加b2是27维

42:30 vector here now in the rules of broadcasting what's going to happen with this bias vector
向量现在在此 以广播规则 将会发生什么 这个偏向矢量

vector
向量 向量
数学与微积分

broadcasting
广播 广播
数学与微积分

42:35 is that this one dimensional vector of 27 will get aligned with an padded dimension of one on
是一个27的维向量 与一个

vector
向量 向量
数学与微积分

dimension
维度 维度
数学与微积分

42:41 the left and it will basically become a row vector and then it will get replicated vertically 32
左键,它将基本上变成一行矢量,然后垂直复制 32

vector
向量 向量
数学与微积分

42:47 times to make it 32 by 27 and then there's an element bias multiply now the question is how do
乘以32乘以27乘32乘以 之后就有一个元素偏差乘以 现在的问题是:

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

42:55 we back propagate from logits to the hidden states the weight matrix w2 and the bias b2
我们返回后从登录向隐藏状态传播 重量矩阵 w2 和偏差 b2

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

matrix
矩阵 矩阵
数学与微积分

43:02 and you might think that we need to go to some matrix calculus and then we have to look up
你也许认为我们需要去一些矩阵积分 然后我们得往上看

matrix
矩阵 矩阵
数学与微积分

矩阵乘法 · 矩阵 43:08

43:08 the derivative for a matrix multiplication but actually you don't have to do any of that and
矩阵乘法的衍生物 但实际上你不必做这些

derivative
导数 导数
反向传播与梯度

matrix
矩阵 矩阵
数学与微积分

43:12 you can go back to first principles and derive this yourself on a piece of paper and specifically what
你可以回到最初的原则 自己用纸上的东西来写出来,具体来说

43:17 I like to do and I what I find works well for me is you find a specific small example that you
我喜欢做,我发现工作对我很好 就是找到一个特别的榜样

43:23 then fully write out and then in the process of analyzing how that individual small example
然后写完之后,然后在分析 如何用个人小例子来说明

43:27 works you will understand the broader pattern and you'll be able to generalize and write out
工作,你会理解更宽的图案 并且可以概括和写出

43:31 the full general formula for what how these derivatives flow in an expression like this
完整通用公式,说明这些衍生物如何在像这样的表达式中流动

expression
表达式 表达式
神经网络基础

43:37 so let's try that out so pardon the low budget production here but what I've done here is I'm
所以让我们试试这个吧 原谅这里预算低的产值 但我在这里所做的是

43:43 writing it out on a piece of paper really what we are interested in is we have a multiply b plus c
写在一张纸上 我们感兴趣的是 我们有乘以b+c

43:48 and that creates a d and we have the derivative of the loss with respect to d and we'd like to
产生d,我们得到了D的损失衍生物 我们希望

derivative
导数 导数
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

43:54 know what the derivative of the loss is with respect to a b and c now these here are a
了解损失的衍生物对b和c的产生物是什么,现在这些是

derivative
导数 导数
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

43:59 little two-dimensional examples of a matrix multiplication two by two times a two by two
矩阵乘法乘以 2乘以 2 乘以 2 次于 2 乘 乘以 3 的二维小示例

matrix
矩阵 矩阵
数学与微积分

44:04 plus a two a vector of just two elements c1 and c2 gives me a two by two now notice here that I have
加上两个矢量, 由两个元素 C1和C2组成, 给了我一个两个两个两个, 现在通知我这里有

vector
向量 向量
数学与微积分

44:13 a bias vector here called c and the bias vector c1 and c2 but as I described over here that bias
这里称为 c 的偏向矢量, 以及 c1和c2 的偏移矢量, 但正如我在这里描述的,

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

vector
向量 向量
数学与微积分

44:20 vector will become a row vector in the broadcasting and will replicate vertically so that's
矢量将成为广播中的行矢量,并将垂直复制,这样

vector
向量 向量
数学与微积分

broadcasting
广播 广播
数学与微积分

44:24 what's happening here as well c1 c2 is replicated vertically and we see how we have two rows of c1
这里发生的事情 还有C1c2 垂直复制 我们可以看到我们是如何有两排C1的

44:30 c2 as a result so now when I say write it out I just mean like this basically break up this
结果是c2 所以现在我说写出来时... ...我的意思是,这基本上就分手了

44:37 matrix multiplication into the actual thing that that's going on under the hood so as a result
将矩阵矩阵乘进到 引擎盖下正在发生的实际事情中,从而产生

matrix
矩阵 矩阵
数学与微积分

44:43 of matrix multiplication and how it works d11 is the result of a dot product between the first
矩阵乘数及其如何工作 d11是第一个

matrix
矩阵 矩阵
数学与微积分

44:48 row of a and the first column of b so a11 b11 plus a12 b21 plus c1 and so on so forth for all the
a行和b 等 a11 b11+a12 b21加c1等第一列

44:59 other elements of d and once you actually write it out it becomes obvious this is just a bunch of
很明显这只是一堆

45:04 multiplies and ads and we know from micrograd how to differentiate multiplies and ads and so this
倍数和广告,我们从微格中知道如何区分多相数和广告。

micrograd
micrograd 自动微分库 micrograd 自动微分库
神经网络基础

45:11 is not scary anymore it's not just matrix multiplication it's just tedious unfortunately
不再可怕了,这不仅仅是矩阵乘法 只是无聊

matrix
矩阵 矩阵
数学与微积分

45:16 but this is completely tractable we have dl by d for all of these and we want dl by
但这是完全可以被牵动的,我们有DD 来做所有这些,我们想要DL

45:22 all these little other variables so how do we achieve that and how do we actually get the
如何实现这一点,我们如何真正得到

45:26 gradients okay so the low budget production continues here so let's for example derive the
梯度可以,所以低预算生产继续在这里进行 所以让我们来举例说

45:32 derivative of the loss with respect to a11 we see here that a11 occurs twice in our simple
与a11有关的损失的衍生物 我们在这里看到A11发生两次

derivative
导数 导数
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

45:38 expression right here right here and influences d11 and d12 so this is so what is dl by d a11
这里的表达式在这里影响 d11 和 d12 所以这就是 d11 的 dl

expression
表达式 表达式
神经网络基础

45:46 well it's dl by d11 times the local derivative of d11 which in this case is just b11 because that's
以 d11 乘以 d11 是 d11 本地衍生的 d11 倍。

derivative
导数 导数
反向传播与梯度

45:55 what's multiplying a11 here so and likewise here the local derivative of d12 with respect to a11
这里乘以a11的值是多少? 这里乘以 11, 这里也乘以 同样这里 d12 的本地衍生物相对于 a11

derivative
导数 导数
反向传播与梯度

46:02 is just b12 and so b12 will in the chain rule therefore multiply dl by d12 and then because
仅是 b12, 所以在链条规则中b12 将会乘以 dl 乘以 d12 , 然后因为

chain rule
链式法则 链式法则
反向传播与梯度

46:09 a11 is used both to produce d11 and d12 we need to add up the contributions of both of those sort of
a11 既用于生产 d11 和 d12, 我们需要将这两种贡献加起来

46:18 chains that are running in parallel and that's why we get a plus just adding up those two
并平行运行的链条 所以加起来

46:24 those two contributions and that gives us dl by d a11 we can do the exact same analysis for the
这两笔捐款 和那给我们Dl DA11

46:30 other one for all the other elements of a and when you simply write it out it's just super simple
单一个元素的所有其他元素, 当你简单地写出来的时候, 它就非常简单

46:36 um taking of gradients on you know expressions like this you find that
平方平方平面的渐变,您知道像这样的表达式,您发现

46:44 this matrix dl by d a that we're after right if we just arrange all of them in the same shape as a
以我们追求正确的方式 把它们都安排成一个

matrix
矩阵 矩阵
数学与微积分

46:51 takes so a is just to write your matrix so dl by d a here will be also just the same shape
所以一个只需要写您的矩阵, 这样一字一字Dl 和DD 这里也会是相同的形状

matrix
矩阵 矩阵
数学与微积分

46:59 uh tensor with the derivatives now so dl by d a11 etc and we see that actually we can express what we've
等我们发现,我们可以表达我们所拥有的

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

47:06 written out here as a matrix multiply and so it just so happens that dl by that all of these formulas
以矩阵乘法写出,所以它就发生了, 所有这些公式都用

matrix
矩阵 矩阵
数学与微积分

47:13 that we've derived here by taking gradients can actually be expressed as a matrix multiplication
我们从这里得出, 使用梯度可以 以矩阵乘法表达

matrix
矩阵 矩阵
数学与微积分

47:19 and in particular we see that it is the matrix multiplication of these two matrices so it
尤其是我们看到,正是这两个矩阵的乘法乘以了矩阵,因此

matrix
矩阵 矩阵
数学与微积分

47:26 is the um dl by d and then matrix multiplying b but b transpose actually so you see that b21 and b12
以 d 为um dl 乘以 d, 然后以矩阵方式乘以 b 但b 实际上转换, 这样您可以看到 b21 和 b12

matrix
矩阵 矩阵
数学与微积分

47:36 have changed place whereas before we had of course b11 b12 b21 b22 so you see that this other matrix
位置已经改变, 而在我们之前当然有 b11 b12 b12 b21 b22 。 所以您可以看到这个其他矩阵

matrix
矩阵 矩阵
数学与微积分

47:45 b is transposed and so basically what we have on story short just by doing very simple reasoning
b 被转换,所以基本上我们的故事简略 仅仅通过做非常简单的推理

47:51 here by breaking up the expression in the case of a very simple example is that dl by d a is which is
这里通过打破表达式的表达方式, 以简单的例子为例, 简单的例子就是 dl by d a 是

expression
表达式 表达式
神经网络基础

47:58 this is simply equal to dl by dd matrix multiplied with b transpose so that is what we have so far
这等于 dl 以dd 矩阵乘以 b 转换, 这样我们就可以得到

matrix
矩阵 矩阵
数学与微积分

48:07 now we also want the derivative with respect to um b and c now for b i'm not actually doing the
现在,我们想要的衍生物 与umb和c有关 现在为b

derivative
导数 导数
反向传播与梯度

48:15 full derivation because honestly it's um it's not deep it's just annoying it's exhausting
完全推断是因为说实话,这是嗯 它不是深的... ...它只是烦人 它累坏了

48:21 you can actually do this analysis yourself you'll also find that if you take this these expressions
也可以自己做这个分析 如果你用这些表达式

48:25 and you differentiate with respect to b instead of a you will find that dl by db is also a
并且区分b项,而不是B项,你会发现dl 逐 db 也是一种

48:31 matrix multiplication in this case you have to take the matrix a and transpose it and matrix
在此情况下, 您必须使用矩阵 a 和矩阵转换

matrix
矩阵 矩阵
数学与微积分

48:37 multiply that with dl by dd and that's what gives you a dl by db and then here for the
乘以 dl 乘以 Dl 乘dd , 这就是给你一个 dl by db 然后在这里

48:44 offsets c1 and c2 if you again just differentiate with respect to c1 you will find an expression
替换 c1和 c2, 如果您再次对 C1 进行区分, 您将会找到表达式

expression
表达式 表达式
神经网络基础

48:50 like this and c2 an expression like this and basically you will find that dl by dc is simply
像这样和 C2 这样的表达式, 基本上你会发现 dl by dc 就是简单的

expression
表达式 表达式
神经网络基础

48:58 because they're just offsetting these expressions you just have to take the dl by dd matrix um
因为它们只是抵消这些表达式 你只需要用 dd 矩阵来取出 dl 数据

matrix
矩阵 矩阵
数学与微积分

49:04 of the derivatives of d and you just have to sum across the columns and that gives you the
和您只需要在列的对齐中进行对比, 这样就可以

49:11 derivatives for c so long story short the backward pass of a matrix multiply is a matrix
长长的c 衍生物短短的矩阵乘数的后空过是一个矩阵

backward pass
反向传播 反向传播
神经网络基础

matrix
矩阵 矩阵
数学与微积分

49:18 multiply and instead of just like we had d equals a times b plus c in a scalar case
乘数而不是像我们一样,在一个标标量的情况下, d 等于 b+ c 乘以乘以

scalar
标量 标量
数学与微积分

49:25 we sort of like arrive at something very very similar but now with a matrix multiplication
我们喜欢达成非常相似的,但现在 有了矩阵乘法

matrix
矩阵 矩阵
数学与微积分

49:29 instead of a scalar multiplication so the derivative of d with respect to a is um
而不是一个斜度乘法,因此d的衍生物相对于a的是一个微米

scalar
标量 标量
数学与微积分

derivative
导数 导数
反向传播与梯度

49:38 dl by dd matrix multiply b transpose and here it's a transpose multiply dl by dd but in both
dl 由 dd 矩阵乘以 b 转换, 这里是一个转换乘以 dd 的 dl 乘以 dD, 但两者都

matrix
矩阵 矩阵
数学与微积分

49:45 cases it's a matrix multiplication with the derivative and the other term in the multiplication
情况是乘法中衍生物和乘法中另一个词的矩阵乘数

matrix
矩阵 矩阵
数学与微积分

derivative
导数 导数
反向传播与梯度

49:53 and for c it is a sum now i'll tell you a secret i can never remember the formulas that we just
和C是一笔总和 现在我要告诉你一个秘密 我永远记不起我们刚刚

49:59 arrived for backpropagating for matrix multiplication and i can back propagate through these expressions
抵达后进行回向插图, 用于矩阵乘法, 我可以返回通过这些表达式传播

matrix
矩阵 矩阵
数学与微积分

50:04 just fine and the reason this works is because the dimensions have to work out uh so let me give
工作的原因是因为维维度必须解决 所以让我给

50:09 you an example say i want to create dh then uh what should dh be number one i have to know that
举个例子,我说我想创造dh 然后呃什么应该成为第一 我必须知道

50:17 the shape of dh must be the same as the shape of h and the shape of h is 32 by 64 and then the other
dh 形状必须和 h 的形状相同, h 的 形状是 32 乘 64 , 然后是另一个

50:24 piece of information i know is that uh dh must be some kind of matrix multiplication of d logits
我知道信息的一部分是,嗯dh 必须是某种矩阵的 dlogits 乘法

matrix
矩阵 矩阵
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

50:30 with w2 and d logits is 32 by 27 and w2 is 64 by 27 there is only a single way to make the shape
与 w2 和 d 相加的 日志为 32 乘 27 和 W 2 乘 64 乘 7 只有一个方法来形状

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

50:40 work out uh in this case and it is indeed the correct result in particular here h needs to be 32 by 64
特别是这里的正确结果... h需要32x64

50:48 the only way to achieve that is to take a d logits and matrix multiply it with uh you see how i have
唯一实现这一点的方法 就是用一个D登录和矩阵 乘以它,呃,你看到我是如何

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

matrix
矩阵 矩阵
数学与微积分

50:56 to take w2 but i have to transpose it to make the dimensions work out so w2 transpose and it's the only
使维维度运作起来 所以W2转换,这是唯一的

51:02 way to make these two matrix multiply those two pieces to make the shapes work out and that turns
使这两个矩阵将这两块的形状乘以 使形状运作起来,然后转过来

matrix
矩阵 矩阵
数学与微积分

51:07 out to be the correct formula so if we come here uh we want dh which is da and we see that da is
来的时候,我们想看看 da是

51:14 dl by dd matrix multiply b transpose so that's d logits multiply and b is w2 so uh w2 transpose
dl 由 dd 矩阵乘以 b 转换, 这样就是 dlogits 乘以 b 是 w2 所以 uh w2 转换

matrix
矩阵 矩阵
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

51:22 which is exactly what we have here so there's no need to remember these formulas uh similarly
这正是我们这里的本事 所以没必要记着这些公式

51:28 now if i want uh dw2 well i know that it must be a matrix multiplication of uh d logits and h
如果我想现在... 嗯dw2 好 我知道它一定是 矩阵乘法的

matrix
矩阵 矩阵
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

51:37 and maybe there's a few transpose like there's one transpose in there as well and i don't know
也许还有几处转体 就像里面也有一处转体 我不知道

51:41 which way it is so i have to come to w2 and i see that its shape is 64 by 27 and that has to come
也就是我必须到W2, 并且我看到它的形状是64乘27, 而这必须来

51:48 from some matrix multiplication of these two and so to get a 64 by 27 i need to take um h i need to
需要从这两个模型的矩阵乘法中 乘以这些,所以要得到64乘27 我需要用到hmh

matrix
矩阵 矩阵
数学与微积分

51:57 transpose it and then i need to matrix multiply it um so that will become 64 by 32 and then i need to
转接它,然后我需要矩阵 乘以它的um 那样就会变成64 乘以32 然后我要

matrix
矩阵 矩阵
数学与微积分

52:04 matrix multiply with the 32 by 27 and that's going to give me a 64 by 27 so i need to
与32乘27乘32乘27 这会给我64乘27 所以我需要

matrix
矩阵 矩阵
数学与微积分

52:09 matrix multiply this with the logits that shape just like that that's the only way to
矩阵将这个乘以成像这样的日志, 这是唯一的方法

matrix
矩阵 矩阵
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

52:13 make the dimensions work out and just use matrix multiplication and if we come here we see that
使维度 工作出来,只是使用矩阵乘法 如果我们来到这里,我们可以看到

matrix
矩阵 矩阵
数学与微积分

52:19 that's exactly what's here so h transpose a for us is h uh multiply with the logits so that's w2 and
这正是这里的特征 所以h转换一个给我们 是Huh乘以日志 所以是W2和W2

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

52:27 then db2 is just the um vertical sum and actually in the same way there's only one way to make
然后Db2只是 um 垂直和,实际上以同样的方式

52:37 the shapes work out i don't have to remember that it's a vertical sum along the zero of axis
形状的形状会起作用 我不需要记住,这是沿轴零线垂直的和

52:41 because that's the only way that this makes sense because b2 shape is 27 so in order to get a um
因为这是唯一合理的方法 因为B2形状是27 所以为了得到一个嗯

52:48 delo jits here is 32 by 27 so knowing that it's just some over delo jits in some direction
这儿的迪罗点点是32比27 27,所以知道它只是一些过头点点 在某个方向

52:59 that direction must be zero because i need to eliminate this dimension so it's this
方向必须是零 因为我需要消除这个维度 所以这是

dimension
维度 维度
数学与微积分

53:06 so this is so this is kind of like the hacky way let me copy paste and delete that
所以这就像黑客式的 复制和删除

53:11 and let me swing over here and this is our backward pass for the linear layer uh hopefully
让我翻过这里,这是我们 向线性层的后退通行证 希望

backward pass
反向传播 反向传播
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

链式法则 · 偏差 53:17

53:17 so now let's uncomment these three and we're checking that we um got all the three derivatives
所以现在让我们取消这三个问题 我们检查一下我们是不是 得到了所有三个衍生物

53:23 correct and uh run and we see that h w2 and b2 are all exactly correct so we back propagate it
正确和呃运行,我们发现hw2和b2都完全正确 所以我们回去传播

53:32 through a linear layer now next up we have derivative for the h already and we need to
经由线性层,现在下一个是直线层 我们已经有了h的衍生物 我们需要

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

derivative
导数 导数
反向传播与梯度

53:40 back propagate through 10h into h preact so we want to derive dh preact and here we have to back
以10小时后向后传播到 h 预过程, 所以我们要生成 dh 预过程。 我们必须返回这里

53:48 propagate through a 10h and we've already done this in micrograd and we remember that 10h is a
10小时后我们已在微研究生中完成 我们记得10小时是

micrograd
micrograd 自动微分库 micrograd 自动微分库
神经网络基础

53:52 very simple backward formula now unfortunately if i just put in d by dx of 10h of x into
简单后退公式 现在不幸地 如果我把D乘以10x的 dx

53:57 volt from alpha it lets us down it tells us that it's a hyperbolic secant function squared of x
它告诉我们,它是一个双曲伸展函数 x 平方

54:03 it's not exactly helpful but uh luckily google image search does not let us down and it gives
并不完全有用 但幸运的谷歌图像搜索 并没有让我们失望,它给了

54:08 us the simpler formula in the particular if you have that a is equal to 10h of z then da by dz
等於 10 z 的,然后按 dz 键

54:15 back propagating through 10h is just 1 minus a square and take note that uh 1 minus a square a
反向传播10小时为1减去一个方形,注意

54:22 here is the output of the 10h not the input to the 10h z so the da by dz is here formulated in
这里显示的是 10 的输出值, 不是 10 z 的输入值, 所以 da 以 dz 表示

54:29 terms of the output of that 10h and here also in google image search we have the full
以10小时输出的值计算, 这里还有谷歌图像搜索中我们拥有的全部

54:34 derivation if you want to actually take the actual definition of 10h and work through the math to
如果你想实际将10小时的实际定义 并贯穿数学到

54:39 figure out 1 minus 10h square of z so 1 minus a square is the local derivative in our case that is 1
minus
以1x10平方公尺乘以1乘以10平方尺,以1乘等于1乘以1是本地衍生物

derivative
导数 导数
反向传播与梯度

54:49 the output of 10h square which here is h so it's h square and that is the local derivative
10h 平方的输出值为 h, 这里是 h, 所以是 h 平方, 这是本地衍生物

derivative
导数 导数
反向传播与梯度

54:57 and then times the chain rule dh so that is going to be our candidate implementation so if we come
然后乘以连锁规则, 从而成为我们候选人执行的环节。 如果我们来的话

chain rule
链式法则 链式法则
反向传播与梯度

implementation
实现 实现
代码与工程

55:03 here and then uncomment this let's hope for the best and we have the right answer okay next up we
并在此不作任何评论 让我们希望最好的 我们有一个正确的答案

55:13 have dh preact and we want to back propagate into the gain the b and raw and the b and bias so here
原 dh 预言和偏向, 我们要回到 B 和 b 和 b, 并在此传播

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

55:19 this is the bash norm parameters b and gain and bias inside the bash norm that take the b and
这是b和b的制片标准 b 以及B中收益和偏见

norm
范数 范数
数学与微积分

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

55:24 raw that is exact unit Gaussian and they scale it and shift it and these are the parameters
等于单位 Gaussian 的单位,它们缩放它并移动它 这些是参数

55:30 of the bash norm now here we have a multiplication but it's worth noting that this multiply is very
这儿我们有一个乘法 但值得指出的是 这个乘法非常

norm
范数 范数
数学与微积分

55:36 very different from this matrix multiply here matrix multiply are dark products between rows
与此矩阵的不同的是, 这里的矩阵乘数是行之间的暗产品

matrix
矩阵 矩阵
数学与微积分

55:41 and columns of these matrices involved this is an element wise multiply so things are quite a bit
以及这些矩阵的列内, 涉及到这个元素是一个明智的因素, 乘法是明智的,所以事情就有点

55:46 simpler now we do have to be careful with some of the broadcasting happening in this line of
现在,我们一定要小心一些 广播在这条线上发生

broadcasting
广播 广播
数学与微积分

55:51 code though so you see how b and gain and b and bias are 1 by 64 but h preact and b and raw are 32 by 64
代码,但这样你可以看到 b和收益,b和偏差是1乘64,h前置和b和原为32乘64

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

56:02 so we have to be careful with that and make sure that all the shapes work out fine and that the
所以我们必须小心谨慎 确保所有的形状都顺利

56:05 broadcasting is correctly back propagated so in particular let's start with db and gain
广播被正确回传,所以特别让我们从 db 开始,然后获得

broadcasting
广播 广播
数学与微积分

56:11 so db and gain should be and here this is again element wise multiply and whenever we have
所以,Db和收益应该存在 在这里,这是再次元素 智慧乘数和我们拥有的

56:18 a times b equals c we saw that the local derivative here is just if this is a the local derivative
a乘以b 等于 c 我们发现这里的本地衍生物 就是如果这是本地衍生物

derivative
导数 导数
反向传播与梯度

56:23 is just the b the other one so the local derivative is just b and raw and then times chain rule
也就是B是另一个b,所以本地衍生物是b和原始的 然后是连锁规则

derivative
导数 导数
反向传播与梯度

chain rule
链式法则 链式法则
反向传播与梯度

56:31 so dh preact so this is the candidate gradient now again we have to be careful because
所以, dh 预言,所以这是候选梯度 现在再次我们必须小心 因为

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

56:40 b and gain is of size 1 by 64 but this here would be 32 by 64 and so the correct thing to do in this
b 和收益是1乘64的大小, 但这里是32乘64 所以正确的做法是这样做

56:50 case of course is that b and gain here is a rule vector of 64 numbers it gets replicated vertically
当然, 情况是 b 和 增益是一条64 个代码矢量的正则矢量, 它会垂直复制

vector
向量 向量
数学与微积分

56:56 in this operation and so therefore the correct thing to do is to sum because it's being replicated
因为它正在被复制

operation
运算 运算
神经网络基础

57:04 and therefore all the gradients in each of the rows that are now flowing backwards need to sum up
因此,现在向后流动的每一行的所有梯度都需要汇总

57:09 to that same tensor db and gain so if you sum across all the zero all the examples basically
同样的抗拉度和收益 所以如果你将所有例子 都加到零

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

57:17 which is the direction which just gets replicated and now we have to be also careful because
这也是刚刚推广的方向, 现在我们也必须小心,因为

57:22 we um b and gain is of shape 1 by 64 so in fact i need to keep them as true otherwise i would
收益是1乘64的形状 所以事实上我需要保持它们的真实性 否则我会

57:30 just get 64 now i don't actually really remember why the b and gain and the b and bias i made them
记得我为什么在B和B上 和偏见下 给他们的

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

57:37 be 1 by 64 um but the biases b1 and b2 i just made them be one dimensional vectors they're not
i 和 b2 的偏差 i 刚刚把它们变成一个维矢量 它们不是

57:45 two dimensional tensors so i can't recall exactly why i left the gain and the bias as two dimensional
分解为两维的抗拉,所以我记不清为什么 我离开得益和偏向是二维

recall [ri'kɔ:l]
召回率 一种分类模型 指标，用于回答以下问题：在所有可能的正类别标签中， 模型正确地识别出了多少个？即：
神经网络基础

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

57:52 but it doesn't really matter as long as you are consistent and you're keeping it the same
但只要你始终一致, 保持不变,这其实并不重要

57:56 so in this case we want to keep the dimension so that the tensor shapes work
所以在这种情况下,我们想保持维度 这样气压就能形成

dimension
维度 维度
数学与微积分

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

58:00 next up we have b and raw so db and raw will be uh b and gain
下一个我们有B和原的,所以Db和原的 将是uhb和收益

58:09 multiplying dh preact that's our chain rule now what about the um dimensions of this we have
乘以 dh 预过程,这是我们现在的 链规则 那么这个维度的微维

chain rule
链式法则 链式法则
反向传播与梯度

58:19 to be careful right so dh preact is 32 by 64 b and gain is 1 by 64 so we'll just get replicated
注意要小心,这样 dh 的预数是 32x 64 b 收益是 1x 64 所以我们只是复制

58:27 and to create this multiplication which is the correct thing because in a forward pass it also
并创建这个乘法,这是正确的事情 因为在一个前传中它也

forward pass
前向传播 前向传播
神经网络基础

58:33 gets replicated in just the same way so in fact we don't need the brackets here we're done and the
以同样的方式复制 所以事实上我们不需要括号了 我们在这里完成

58:38 shapes are already correct and finally for the bias very similar this bias here is very very
形状已经正确, 最终也是正确的, 因为偏见非常相似。 此偏向非常非常相似

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

58:45 similar to the bias we saw in the in the linear layer and we see that the gradients from
类似我们所看到的线性层的偏差 我们发现

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

58:51 hpreact will simply flow into the biases and add up because these are just these are just
偏差会直接进入偏向中,加起来,因为这些只是

58:55 offsets and so basically we want this to be dh preact but it needs to sum along the right dimension
抵消,所以基本上我们想要这个是 dh 预言 但是它需要按照正确的方面来总结

dimension
维度 维度
数学与微积分

59:03 and in this case similar to the gain we need to sum across the zero dimension the examples
和我们从零维中需要的收益相似 举例

dimension
维度 维度
数学与微积分

59:08 because of the way that the bias gets replicated very quickly and we also want to have to keep them
因为我们的偏见很快被复制, 我们也想保留它们

bias ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

59:13 is true and so this will basically take this and sum it up and give us a 1 by 64
事实如此,所以这基本上 将采取这个和总结起来 给我们一个比64

59:19 so this is the candidate implementation it makes all the shapes work let me bring it up
所以这是候选人的落实 使所有形状都运作起来 让我把它提上来

implementation
实现 实现
代码与工程

59:27 down here and then let me uncomment these three lines to check that we are getting the correct result
让我先在这里下面 然后再解开这三行的音调 检查我们是否得到了正确的结果

59:33 for all the three tensors and indeed we see that all of that got back propagated correctly so now
三个10万的三百万,事实上我们看到 所有那些返回的都正确地传播了

59:39 we get to the batch norm layer we see how here being gain and being bias are the parameters so
我们看到,我们是如何获得和偏向的参数是

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

norm
范数 范数
数学与微积分

59:45 the back propagation ends but b and raw now is the output of the standardization so here what I'm
后背传播结束,但B和原是标准化的输出 所以这里我是什么

back propagation
反向传播 反向传播
神经网络基础

59:52 doing of course is I'm breaking up the batch norm into manageable pieces so we can back propagate
当然,我把批量规范 拆成可以管理的碎片 这样我们就可以回源传播

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

norm
范数 范数
数学与微积分

59:56 through each line individually but basically what's happening is b and mean i is the sum
每一个线的每个线 单个但基本上发生的事情是 B和意味着我就是总

1:00:03 so this is the b and mean i I apologize for the variable naming
所以这是B和意指 我为变数命名道歉

1:00:08 b and diff is x minus mu b and if to is x minus mu squared here inside the variance
b和 diff 是 x 减去 mub, 如果是 x 减去 mu 则是 x 减 mu 方形, 这里在差异内

1:00:16 b and var is the variance so sigma square this is b and var and it's basically the sum of squares
b和var是差异 所以Sigma 方形 这是b和var 基本上就是平方之和

1:00:25 so this is the x minus mu squared and then the sum now you'll notice one departure here
所以这是x减去 musqud 然后是总和 现在你会注意到一个出发

1:00:32 here it is normalized as one over m which is the number of examples here I am normalizing as one over

这里它被正常化为一/m, 也就是这里的示例数量, 我在这里正在将一个正常化为一/i

1:00:38 n minus one instead of m and this is deliberate and I'll come back to that in a bit when we are at

而不是M,这是故意的 当我们到了的时候我会再回到那个

1:00:43 this line it is something called the bestial correction but this is how I want it in our case

这条线,这是一种叫兽用矫正的仪式 但是这就是我想要的

1:00:51 b and var in then becomes basically b and var plus epsilon epsilon is y negative five

b 和 var 在那时变成 b 基本b 和 vv 加 Epsilon epsilon 是 y 负 5

1:00:57 and then it's one over square root is the same as raising to the power of negative point five

和它一平方根相同 等于向负点5的功率上升

1:01:03 right because point five is squared and then negative makes it one over square root

右对是因为五点是平方,然后负后变为一平方根

1:01:08 so b and var nth is a one over this denominator here and then we can see that b and raw which is

b和var nth是这个分母的一面 然后我们可以看到

1:01:15 the x hat here is equal to the b and diff the numerator multiplied by the b and var nth

x 顶在此等于 b和 diff 乘以 b 和 var nth

1:01:24 and this line here that creates pre-h pre-act was the last piece we've already back propagated through it

这条创造h前预反应的线 是我们已经通过它 传播到这里的最后一块

1:01:31 so now what we want to do is we are here and we have b and raw and we have to first back propagate

所以我们现在要做的是 我们在这里,我们有b和b 生,我们必须首先回传播

1:01:37 into b and diff and b and var nth so now we're here and we have d b and raw and we need to back

和Diff,b,b,var nth 所以现在我们在这里 我们有db,b和ww

1:01:44 propagate through this line now I've written out the shapes here and indeed b and var nth is

透过这条线传播,我现在写了这里的形状, 而事实上B和varnth是

1:01:51 a shape one by 64 so there is a broadcasting happening here that we have to be careful with

形状64,64,所以有一个广播 这里发生 我们必须小心

broadcasting
广播 广播
数学与微积分

1:01:56 but it is just an element wise simple multiplication by now we should be pretty comfortable with that
但这只是个明智的简单乘法因素 现在我们应该可以接受

1:02:01 to get d b and diff we know that this is just b and var nth multiplied with d b and raw
来获得 db和 diff 我们知道这只是 b和 var nth 乘以 d b 和 raw

1:02:11 and conversely to get d b and var nth we need to take b and diff and multiply that by d b and
反过来,为了得到db和vvnth 我们需要采取b和diff 乘以db和

1:02:19 raw so this is the candidate but of course we need to make sure that broadcasting is obeyed so in
这也是候选人,当然要保证广播被听

broadcasting
广播 广播
数学与微积分

1:02:27 particular b and var nth multiplying with d b and raw will be okay and give us 32 by 64 as we expect
和量子方xxxx(以 db和 b和 原为单位) 将可以,并按我们预期的 给我们32乘64

1:02:36 but d b and var nth would be taking a 32 by 64 multiplying it by 32 by 64 so this is a 32 by
但 db 和 var nth 将乘以32 乘以64 乘以34 乘以32乘以64 所以这个是32

1:02:45 64 but of course d b this b and var nth is only 1 by 64 so the second line here needs a sum across the
和nth的1乘64 所以这里第二行需要一整笔

1:02:55 examples and because there's this dimension here we need to make sure that keep them is true so
this
举例,因为这里有这个维度 我们需要确保它们是真的

dimension
维度 维度
数学与微积分

1:03:02 is the candidate let's erase this and let's swing down here and implement it and then let's comment
候选人是,让我们删除这个 然后我们在这里摇摆一下 执行它,然后我们评论

1:03:11 out d b and var nth and d b and diff now we'll actually notice that d b and diff by the way
和db和dvar nth, db和diff 现在我们会发现db和 diff

1:03:19 is going to be incorrect so when i run this b m var nth is correct b and diff is not correct
将会不正确, 所以当我运行 b m var nth 时, b 是正确的 diff 是不正确的

1:03:28 and this is actually expected because we're not done with b and diff so in particular when
而这其实是预期的,因为我们没有完成B和diff,特别是当

1:03:34 we slide here we see here that b and raw is a function of b and diff but actually b and var nth
我们在这里滑动,我们可以看到这里的 b和 row 是b和 diff 的函数, 但实际上 b和 var nth

1:03:40 is a function of b and var which is a function of b and diff do which is a function of b and diff
b和 var 函数是 b和 diff 函数是b和 diff do 函数是 B和 diff

1:03:45 so it comes here so b d and diff um these variable names are crazy i'm sorry it branches out into two
所以这里出现,所以b d和diff um 这些变异名字很疯狂 我很抱歉它分成两个

1:03:52 branches and we've only done one branch of it we have to continue our back propagation and
我们只做了一个分支 我们必须继续背后的传播

back propagation
反向传播 反向传播
神经网络基础

1:03:57 eventually come back to b and diff and then we'll be able to do a plus equals and get the
最终回到b和diff 然后我们就可以做一个加+等价的

1:04:01 actual correct gradient for now it is good to verify that c m p also works it doesn't just lie
现在,现在可以核实 cm p 是否有效,而不只是谎言

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:04:06 to us and tell us that everything is always uh correct it can in fact detect when your gradient
告诉我们一切都总是正确, 能够检测到

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:04:11 is not correct so it's that's good to see as well okay so now we have the derivative here and we're
是不正确的 所以这是很好看到也没关系 所以我们现在有衍生物在这里

derivative
导数 导数
反向传播与梯度

1:04:16 trying to back propagate through this line and uh because we're raising to a power of negative
试图回回 透过这条线传播 和呃,因为我们的 成长到负的力量

1:04:21 point five i brought up the power rule and we see that basically we have that d b and var
5点5 我提出权力规则 我们发现基本上我们有DB和Var

1:04:26 will now be we bring down the exponent so negative point five times uh x which is this
现在,我们将把 负点5倍x

1:04:34 and now raised to the power of negative point five minus one which is negative one point five
现升至负5减1等于负1 5等于负1

1:04:39 now we would have to also apply a small chain rule here in our head uh because we need to take
现在我们也不得不在我们的头上 运用一条小链条规则 因为我们需要

chain rule
链式法则 链式法则
反向传播与梯度

1:04:45 further derivative of uh b and var with respect to this expression here inside the bracket but because
和 var 的衍生物

derivative
导数 导数
反向传播与梯度

expression
表达式 表达式
神经网络基础

1:04:51 this is an element wise operation and everything is fairly simple that's just one and so there's
这是一个明智的操作元素 一切都很简单,只是其中的一个

operation
运算 运算
神经网络基础

1:04:56 nothing to do there so this is the local derivative and then times the global derivative
在那里没有做任何事情,所以这是本地衍生品,然后是全球衍生品的倍数

derivative
导数 导数
反向传播与梯度

1:05:01 to create the chain rule this is just times d b and var so this is our candidate let me bring this down
来创造链条规则 这是Db和var的次数

chain rule
链式法则 链式法则
反向传播与梯度

1:05:10 and uncomment the check and we see that we have the correct result now before we back
并取消检查,我们看到 我们得到正确的结果 现在在我们回来之前

1:05:18 propagate through the next line i wanted to briefly talk about the note here where i'm
透过下一行传播 我想简短地谈谈 笔记在这里,我在哪里

1:05:21 using the bestest correction dividing by n minus one instead of dividing by n when i
使用最优校正除以 n 减 1/, 而不是在i 下除以 n

1:05:26 normalize here the sum of squares now you'll notice that this is the departure from the
这儿的平方和正方和 正常化, 现在你会注意到 这是偏离

推断 · 训练 1:05:31

1:05:31 paper which uses one over n instead not one over n minus one there m is rn and um so it turns out
使用一牛n以上而不是一牛n减去一牛1,它有撕和um 所以结果

1:05:39 that there are two ways of estimating variance uh off an array one is the biased estimate which is
有两个方法可以估计差异 额数组的差别

1:05:46 one over n and the other one is the unbiased estimate which is one over n minus one now
一比零,另一个是无偏的估算值 现在一比零减一

1:05:51 confusingly in the paper this is uh not very clearly described and also it's a detail that
报纸上混淆不清... ...这个... ...嗯,没有非常清楚描述 而且它是一个细节

1:05:56 kind of matters i think um they are using the biased version training time but later when
我认为他们使用偏颇版本的训练时间,

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:06:02 they are talking about the inference they are mentioning that when they do the inference
他们谈论的是他们的推理,他们只说:当他们判决的时候,

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

1:06:06 they are using the unbiased estimate which is the n minus one version in um basically for inference
使用不偏直的估计值,即微米中正减1版本的n-mum,基本用于推断

inference ['infərəns]
推断 在机器学习中，推断通常指以下过程：通过将训练过的模型应用于无标签样本 来做出预测。在统计学中，推断是指在某些观测数据条件下拟合分布参 数的过程。（请参阅维基百科中有关统计学推断的文章 (https://en.wikipedia.org/wiki/Statistical_inference)。）
神经网络基础

1:06:15 and to calibrate the the running mean and running variance basically and so they they
并校准运行中和变化的平均值, 基本地使它们

1:06:21 actually introduce a train test mismatch where in training they use the biased version and in the
培训中他们使用偏偏的版本,

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:06:25 and test time they use the unbiased version i find this extremely confusing you can read more about
测试时间,他们使用不偏的版本。 我发现这个非常混乱的您可以读更多关于

1:06:31 the bestest correction and why uh dividing by n minus one gives you a better estimate of the variance
最准确的更正,为什么 uh除以 n减去 1 使你对差异的更精确估计

1:06:36 in a case where you have population size it or samples for a population uh there are very small
在一个情况下,如果有人口规模,或者为某个人口样本,呃,非常小

1:06:42 and that is indeed the case for us because we are dealing with many patches and these mini
对我们来说,这确实是事实,因为我们处理着许多补丁和这些细微的薄膜。

1:06:47 matches are a small sample of a larger population which is the entire training set and so it
火柴是人数众多的样本, 整个训练组就是这个样本

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:06:53 just turns out that uh if you just estimate it using one over n that actually almost always
只不过结果... 如果你只是估计一下它使用一个比N的, 实际上几乎总是

1:06:58 underestimates the variance and it is a biased estimator and it is advised that you use the
它是一个有偏见的估算者,建议使用

1:07:02 unbiased version and divide by n minus one and you can go through this article here that i liked
公正版本,除以零减一,你就可以在这里读读我最喜欢的文章

1:07:07 that actually describes the full reasoning and i'll link it in the video description
描述整个推理 我会在视频描述中将其链接

1:07:12 now when you calculate torshtat variance you'll notice that they take the unbiased flag
当你现在计算 压力变化时 你会注意到他们拿了无偏的旗帜

1:07:17 whether or not you want to divide by n or n minus one uh confusingly they do not um
是否您想要除以 n 或 n 减去 uh , 令人困惑的是他们不 um

1:07:22 mention what the default is for unbiased but i believe unbiased by default is true
提到默认是没有偏见的,但我认为默认是没有公正性的

1:07:28 i'm not sure why the docs here don't cite that now in the bachelor one d the documentation again
我不知道为什么这里的医生 不引用 现在在单身汉的单D文件

1:07:35 is kind of wrong and confusing it says that the standard deviation is calculated via the
标准偏离是用

1:07:39 biased estimator but this is actually not exactly right and people have pointed out that it is not
人民指出,这并非

1:07:44 right in a number of issues since then because actually the rabbit hole is deeper
在那之后,在一系列问题上右转 因为实际上兔子洞更深

1:07:49 and they follow the paper exactly and they use the biased version for training but when they're
并且他们完全按照文件的准确 并使用偏见版本 训练,但当他们

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:07:55 estimating the running standard deviation we are using the unbiased version so again there's the
估计我们使用不偏版的运行标准差

1:08:00 train test mismatch so long story short i'm not a fan of train test discrepancies i basically
火车测试不匹配,长长短短的短短 我才不喜欢火车测试差异

1:08:06 kind of consider um the fact that we use the biased version the training time and the unbiased
考虑一下我们使用偏颇的版本 培训时间和公正

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:08:12 test time i basically consider this to be a bug and i don't think that there's a good reason
测试时间 我基本上认为这是个虫子 我不认为有很好的理由

1:08:16 for that um it's not really they don't really go into the detail of the reasoning behind it
因为嗯,其实不是他们真的 不真正去 详细到背后的推理

1:08:20 in this paper so that's why i basically prefer to use the bestest correction in my own work
本文中写到,所以我更喜欢用 我工作中最精细的校正

1:08:26 unfortunately bachelor does not take a keyword argument that tells you whether or not um you
不幸的是单身汉没有接受关键词的争论 来告诉你是否

1:08:31 want to use the unbiased version of the biased version in both training test and so therefore
想要在训练考试中,

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:08:36 anyone using batch normalization basically in my view has a bit of a bug in the code um
使用批量正常化的人 基本上在我看来 代码里有一点点虫

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:08:41 and this turns out to be much less of a problem if your batch many batch sizes are a bit larger
如果分批量的大小更大, 问题就更少了

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:08:47 but still i just find kind of uh unpalatable so maybe someone can explain why this is okay
但我还是觉得有点不愉快 所以也许有人可以解释 为什么这样没关系

1:08:52 but for now i prefer to use the unbiased version consistently both during training and at test
但目前我更倾向于在培训和测试期间始终使用不偏的版本。

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

召回率 · 反向传播 1:08:57

1:08:57 time and that's why i'm using one over n minus one here okay so let's now actually backprop gate
所以我才要用一个以上零减一个 好,现在让我们来真的后门

backprop
反向传播 反向传播
反向传播与梯度

1:09:03 through this line so the first thing that i always like to do is i like to scrutinize the
透过这条线,所以我最喜欢做的第一件事 就是仔细检查

1:09:09 shapes first so in particular here looking at the shapes of what's involved i see that b and var
形状先形状,所以这里特别的 查看其中所涉及的形状 我看到了B和Var

1:09:15 shape is 1 by 64 so it's a row vector and b and if 2 dot shape is 32 by 64 so clearly here we're
形状为 1x 64 的形状是 1 乘 64, 所以它是行矢量和 b。 如果 2点形状是 32 乘 64 则此表显示

vector
向量 向量
数学与微积分

1:09:23 doing a sum over the 0th axis to squash the first dimension of uh of the shapes here using a sum
以一个和,在0轴上做一个总和,以用一个和来压压形状的 uh 第一个维度

dimension
维度 维度
数学与微积分

1:09:33 so that right away actually hints to me that there will be some kind of a replication or
立刻就向我暗示 会有某种复制或复制

1:09:37 broadcasting in the backward pass and maybe you're noticing the pattern here but basically
播放后传的后传, 也许你注意到这里的模式,但基本上

broadcasting
广播 广播
数学与微积分

backward pass
反向传播 反向传播
神经网络基础

1:09:42 anytime you have a sum in the forward pass that turns into a replication or broadcasting in the
任何时间,您可在前方的通行证中找到一笔金额,该金额可转换为在

forward pass
前向传播 前向传播
神经网络基础

broadcasting
广播 广播
数学与微积分

1:09:47 backward pass along the same dimension and conversely when we have a replication or a
反向,当我们复制或复制时

backward pass
反向传播 反向传播
神经网络基础

dimension
维度 维度
数学与微积分

1:09:53 broadcasting in the forward pass that indicates a variable reuse and so in the backward pass
在前传口中播放显示可变再利用的复选表,在后传口中如此

broadcasting
广播 广播
数学与微积分

forward pass
前向传播 前向传播
神经网络基础

1:09:59 that turns into a sum over the exact same dimension and so hopefully you're noticing that duality
变成一个总数 在同一维度上 所以希望你注意到

dimension
维度 维度
数学与微积分

1:10:04 that those two are kind of like the opposite of each other in the forward and backward pass
这两样人,在前行和后行之间,彼此是相对的。

backward pass
反向传播 反向传播
神经网络基础

1:10:09 now once we understand the shapes the next thing i like to do always is i like to look at
一旦我们了解了形状 我接下来喜欢做的事 总是喜欢看

1:10:13 the toy example in my head to sort of just like understand roughly how the variable dependencies
在我的脑海中,一个玩具的例子 似乎就像大致理解了不同的依赖关系

1:10:18 go in the mathematical formula so here we have a two-dimensional array at b and f2
以数学公式进行,所以这里我们有一个二维阵列,在 b和 f2

1:10:25 which we are scaling by a constant and then we are summing vertically over the columns
以一个常数缩放 然后我们垂直地向列上滚动

scaling ['skeiliŋ]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

1:10:31 so if we have a two by two matrix a and then we sum over the columns and scale
以两个两个矩阵a,然后以栏和称来计算。

matrix
矩阵 矩阵
数学与微积分

1:10:35 we would get a row vector b1 b2 and b1 depends on a in this way where it's just sum that's scaled of a
我们得到一行矢量b1 b2和b1 取决于这样的方式, 它只是一个总和的缩放

vector
向量 向量
数学与微积分

1:10:43 and b2 in this way where it's the second column summed and scaled and so looking at this basically
和b2,这样一来,它是第二列的汇总和缩放, 所以看这个基本上

1:10:52 what we want to do now is we have the derivatives on b1 and b2 and we want to back
我们现在要做的是 我们把B1和b2上的衍生物 放在B1和B2上,我们想回去

1:10:55 propagate them into a's and so it's clear that just differentiating in your head the local derivative
把它们传播到一个的,所以很明显 只是区分在你的头上

derivative
导数 导数
反向传播与梯度

1:11:01 here is one over n minus one times one for each one of these a's and basically the derivative of b1
这里的每个这些a的, 基本上就是 b1 的衍生物, 等于 零乘以 1

derivative
导数 导数
反向传播与梯度

1:11:10 has to flow through the columns of a scaled by one over n minus one and that's roughly what's
需要通过一个平整的列 以一负一乘以一

1:11:17 happening here so intuitively the derivative flow tells us that db and f2 will be the local derivative
直接发生在这里,所以直觉地 衍生物流告诉我们,Db和F2 将是本地的衍生物

derivative
导数 导数
反向传播与梯度

1:11:27 of this operation and there are many ways to do this by the way but i like to do something like this
并且有很多方法可以做这个操作, 但我喜欢做这样的事

operation
运算 运算
神经网络基础

1:11:31 torshtot one's like of b and f2 so i'll create a large array two-dimensional of ones
刻度像b和f2 所以我会创造出一个巨大的阵列二维的

1:11:39 and then i will scale it so 1.0 divide by n minus one
然后我会将它缩放成 1. 0 除以 n 减 1

1:11:43 so this is a array of um one over n minus one and that's sort of like the local derivative
所以这是一阵um 1 以上 n 减去 n 的数列, 这有点像本地衍生物

derivative
导数 导数
反向传播与梯度

1:11:50 and now for the chain rule i will simply just multiply it by db and bar and notice here what's
现在,对于链规则 我只需要把它乘以DB和酒吧 注意这里什么

chain rule
链式法则 链式法则
反向传播与梯度

1:11:59 going to happen this is 32 by 64 and this is just 1 by 64 so i'm letting the broadcasting do the
以后这台是32比64 这是3乘64,这是1比64 所以我让广播机来做

broadcasting
广播 广播
数学与微积分

1:12:06 replication because internally in pi torch basically db and bar which is 1 by 64 row vector
因为在内部的 pi 火炬中, 基本上 db 和 Bar , 也就是 1x 64 行矢量

vector
向量 向量
数学与微积分

1:12:13 will in this multiplication get um copied vertically until the two are of the same shape and then there
将在此乘法中进行垂直复制, 以 um 格式复制到两个形状相同 然后显示

1:12:19 will be an element wise multiply and so that so that the broadcasting is basically doing the
将是一个明智的乘数因素,因此广播基本上就是在进行

broadcasting
广播 广播
数学与微积分

1:12:23 replication and i will end up with the derivatives of db and f2 here so this is the can't date
并进行复制, 我将在此使用 db 和 f2 的衍生物, 所以无法在此日期

1:12:31 solution let's bring it down here let's uncomment this line where we check it and let's hope for
解决之道,让我们把它带下来 让我们取消这条线的音调 我们检查它,让我们希望

1:12:38 the best and indeed we see that this is the correct formula next up let's differentiate here into b and
最好,而且我们确实看到 这是正确的公式 下一个上让我们在这里区分

1:12:44 df so here we have that b and f is element wise squared to create b and f2 so this is a relatively
fb和f是明智的方格元素 来创建 b和f2 所以这是一个相对的

1:12:52 simple derivative because it's a simple element wise operation so it's kind of like the scalar case
简单的衍生物,因为它是一个简单元素的明智操作 所以它有点像标标案

derivative
导数 导数
反向传播与梯度

operation
运算 运算
神经网络基础

1:12:57 and we have that db and f should be uh if this is x squared then the derivative of this is
我们的 db和f应该应该是uh 如果这是x方方形

derivative
导数 导数
反向传播与梯度

1:13:02 2x right so it's simply 2 times b and f that's the local derivative and then times chain rule and the
2x右2x右2乘对,所以它只是 本地衍生物的2倍b和f

derivative
导数 导数
反向传播与梯度

chain rule
链式法则 链式法则
反向传播与梯度

1:13:10 shape of these is the same they are of the same shape so times this so that's the backward pass for
这些形状的形状是相同的形状, 形状相同, 所以在这样的形状下, 此时此时间, 这就是后方的通道

backward pass
反向传播 反向传播
神经网络基础

1:13:17 this variable let me bring it down here and now we have to be careful because we already calculated
这个变量让我把它带下来 现在我们必须小心 因为我们已经计算过了

1:13:22 db and f right so this is just the end of the other uh you know other branch coming back to
和Db和F右右,所以这只是结束 其他呃你知道另一个分支

1:13:29 b and f because b and f will already back propagate it to way over here from b and raw so we now completed
b和f 因为b和f会已经 将它从b和b 和f传播到这里的很远的地方

1:13:37 the second branch and so that's why i have to do plus equals and if you recall we had an incorrect
第二分支 所以这就是为什么我必须做加加等同 如果你记得我们有一个错误

recall [ri'kɔ:l]
召回率 一种分类模型 指标，用于回答以下问题：在所有可能的正类别标签中， 模型正确地识别出了多少个？即：
神经网络基础

1:13:43 derivative for b and f before and i'm hoping that once we append this last missing piece
之前b和f的衍生物 我希望一旦我们附上 最后这一段遗漏

derivative
导数 导数
反向传播与梯度

1:13:48 that we have the exact correctness so let's run and b and f to uh b and f now actually shows the
准确无误的正确性 所以让我们跑,b和f到f到uhb和f 现在实际上显示

1:13:55 exact correct derivative um so that's comforting okay so let's now back propagate through this line
准确无误的衍生物,嗯 所以这很让人放心 好了,让我们现在回到这里 通过这条线传播

derivative
导数 导数
反向传播与梯度

1:14:01 here um the first thing we do of course is we check the shapes and i wrote them out here
我们首先当然要做的是检查形状 我写在这里

1:14:07 and basically the shape of this is 32 by 64 hpbn is the same shape but b and mi is a row vector
基本上,它的形状是 32x 64 hpbn 是相同的形状, 但 b和 mi 是行矢量

vector
向量 向量
数学与微积分

1:14:14 1 by 64 so this minus here will actually do broadcasting and so we have to be careful with
1664,所以这里减去这个,实际上就是广播, 所以我们必须小心

broadcasting
广播 广播
数学与微积分

1:14:19 that and as a hint to us again because of the duality a broadcasting and a forward pass means
并再次向我们示意,因为广播和前方的双重手段。

broadcasting
广播 广播
数学与微积分

forward pass
前向传播 前向传播
神经网络基础

1:14:25 variable reuse and therefore there will be a sum in the backward pass so let's write out the backward
变变量再利用,因此后传通行证中会有一笔总和,所以让我们写出后向

backward pass
反向传播 反向传播
神经网络基础

1:14:31 pass here now um back propagate into the hpbn because this is these are the same shape then
此处的 pass 现在, um back 向hpbn 传播到 hpbn 因为这是相同的形状

1:14:39 the local derivative for each one of the elements here is just one for the corresponding element
这里每个元素的本地衍生物 仅对应元素的

derivative
导数 导数
反向传播与梯度

1:14:43 in here so basically what this means is that the gradient just simply copies it's just a variable
基本上,这里的意思是 梯度只是复制它只是一个变量

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:14:50 assignment it's quality so i'm just going to clone this tensor just for safety to create an exact copy
质量是高质量的 所以我要克隆这个高压器 安全地制作一个精确的复制件

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:14:56 of um db and f and then here to back propagate into this one what i'm inclined to do here is
之后我打算在这里做

1:15:04 dbn mi will basically be uh what is the local derivative well it's negative torch dot one's like
当地衍生品井是负火炬点

derivative
导数 导数
反向传播与梯度

1:15:16 of the shape of uh b and f right and then times the um derivative here db and f and this here
和这个

derivative
导数 导数
反向传播与梯度

1:15:33 is the back propagation for the replicated b and mi so i still have to back propagate through the
即是复制的 b和 mi 的背传播, 所以我仍然必须返回 通过

back propagation
反向传播 反向传播
神经网络基础

1:15:41 replication in the broadcasting and i do that by doing a sum so i'm going to take this whole thing
广播的复制 我做这个的时候做了一个总和 所以我要把这整个事情

broadcasting
广播 广播
数学与微积分

1:15:45 and i'm going to do a sum over the zero dimension which was the replication
我要对零维做一个总和 也就是复制

dimension
维度 维度
数学与微积分

1:15:53 so if you scrutinize this by the way you'll notice that this is the same shape as that
如果你仔细看这个,你会注意到 这个形状和那个一样

1:15:58 and so what i'm doing what i'm doing here doesn't actually make that much sense because it's just a
所以我现在在这里做的事 并没有真正说得那么有道理 因为这只是个

1:16:03 array of ones multiplying db and f so in fact i can just do this um and that is equivalent
等量的平面

1:16:12 so this is the candidate backward pass let me copy it here and then let me comment out
所以这是候选人的后退通行证 让我抄一下这里 然后让我评论

backward pass
反向传播 反向传播
神经网络基础

1:16:19 this one and this one enter and it's wrong damn actually sorry this is supposed to be wrong
而这一个和这个进,这是错的 糟糕的是,这其实是错的

1:16:31 and it's supposed to be wrong because we are back propagating from a bndf into hprebn
因为我们又从一个小牛头 向Hprebn传播了

1:16:38 and but we're not done because b and mi depends on hprebn and there will be a second portion of
但由于B和MI依赖hprebn 而还有第二部分

1:16:44 that derivative coming from this second branch so we're not done yet and we expect it to be incorrect
第二分支的衍生物 我们还没完成,我们期望它不正确

derivative
导数 导数
反向传播与梯度

1:16:49 so there you go uh so let's now back propagate from uh b and mi into hprebn
所以,你去那里嗯所以让我们现在回来 宣传从uhb和mi 向hprepbn

召回率 · 嵌套 1:16:57

1:16:57 and so here again we have to be careful because there's a broadcasting along
所以在这里我们必须小心 因为有广播

broadcasting
广播 广播
数学与微积分

1:17:01 there's a sum along the zero dimension so this will turn into broadcasting in the backward pass now
零维上有一个总和 所以现在这个会变成后空的广播

dimension
维度 维度
数学与微积分

broadcasting
广播 广播
数学与微积分

1:17:06 and i'm going to go a little bit faster on this line because it is very similar to the line that we
我将走得更快一点 因为这条线非常相似

1:17:11 had before and multiple lines in the past in fact so dhprebn will be uh the gradient will be scaled
之前有多条线,过去也有多条线 事实上这样dhprepbn就是uh

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:17:21 by one over n and then basically this gradient here on db and mi is going to be scaled by one
乘以 n 乘以一,然后基本上这个在 db和 mi 上的梯度将缩放为一个

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:17:29 over n and then it's going to flow across all the columns and deposit itself into dhprebn
将它从所有柱子流过 然后沉入 dhprepbn

1:17:35 so what we want is this thing scaled by one over n let me put the constant up front here so scale
所以我们想要的就是 以一比一的牛牛牛身缩小一点 让我把常数放在前面

1:17:46 down the gradient and now we need to replicate it across all the um across all the rows here
向下斜度 现在我们需要复制它 在所有的平面上

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:17:53 so we i like to do that by torch dot ones like of basically um hprebn and i will let broadcasting
所以我们喜欢这样,通过点火烧 诸如基本上umhprbn这样的点 我会让广播

broadcasting
广播 广播
数学与微积分

1:18:05 do the work of um replication so so this is the hprebn and hopefully we can plus equals that
做微微复制的工作 所以,这是hprebn 希望我们能加

1:18:27 so this here is broadcasting um and then this is the scaling so this should be correct
所以这里是广播 嗯,然后这是缩放,所以这应该是正确的

broadcasting
广播 广播
数学与微积分

scaling ['skeilin]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

1:18:33 okay so that completes the back propagation of the bastram layer and we are now here let's back
完成肌肉层的后传播 我们现在在这里 让我们回到

back propagation
反向传播 反向传播
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:18:39 propagate through the linear layer one here now because everything is getting a little
透过线性层一传播 因为一切都变得有点

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:18:44 vertically crazy i copy pasted the line here and let's just back propagate through this one line
我抄过这条线,我这复制了一条线 我们回到这条线上 透过这条线传播

1:18:50 so first of course we inspect the shapes and we see that this is 32 by 64 mcat is 32 by 30
所以首先我们当然要检查形状 我们看到这是32比64 30

1:18:58 w1 is 30 by 64 and b1 is just 64 so as i mentioned back propagating through linear layers is fairly
30x64,b1等于64 所以,我提到后方的传播 通过线性层是公平的

1:19:07 easy just by matching the shapes so let's do that we have that d mcat should be um some
简单,只要匹配形状,让我们做 我们有DMCT应该是嗯

1:19:15 matrix multiplication of dhprebn with uh w1 and one transpose thrown in there so to make
将dhprebn和uh w1乘以一个转体投入其中,以便制造

matrix
矩阵 矩阵
数学与微积分

1:19:24 uh mcat be 32 by 30 i need to take uh dhprebn 32 by 64 and multiply it by w1 that transpose
30x32,需要32号德赫普林32号64乘以1

1:19:39 to get d only one i need to end up with 30 by 64 so to get that i need to take uh mcat transpose
才能得到一个我需要的 30x64,然后30x64 所以,我需要用 呃,mcat变换

1:19:50 and multiply that by uh dhprebn and finally to get db1 this is a addition and we saw that basically
乘以 uh dhprebn 最终得到 Db1 这是添加

1:20:05 i need to just sum the elements in dhprebn along some dimension and to make the dimensions work
使这些维度发挥作用

dimension
维度 维度
数学与微积分

1:20:11 out i need to sum along the zero axis here to eliminate this dimension and we do not keep dims
消除这个维度 我们不会保持暗淡

dimension
维度 维度
数学与微积分

1:20:18 uh so that we want to just get a single one dimensional vector of 64 so these are the claimed
derivatives
以便我们只得到64度的 单一维矢量,所以这些是声称的衍生物

vector
向量 向量
数学与微积分

1:20:27 let me put that here and let me uncommon three lines and cross our fingers everything is great
让我把它放在这里 让我用不寻常的三行三行 交叉手指,一切都很好

1:20:35 okay so we now continue almost there we have the derivative of mcat and we want to derivative
所以我们现在继续 几乎在那里,我们有 衍生的毛猫

derivative
导数 导数
反向传播与梯度

1:20:40 we want to backpropagate into m so i again copied this line over here
我们要将软件后方的 pracate 转换到 m, 所以我再次复制了这条线在这里

1:20:46 uh so this is the forward pass and then this is the shapes so remember that the shape here was
这是前方的通道 然后是形状 所以记住这里的形状

forward pass
前向传播 前向传播
神经网络基础

1:20:51 32 by 30 and the original shape of m was 32 by 3 by 10 so this layer in the forward pass as you
32乘32乘30乘32乘32乘 30, 最初的米形状是32乘3乘10 所以前方的这个层和你一样

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

forward pass
前向传播 前向传播
神经网络基础

1:20:58 recall that the concatenation of these three 10 dimensional character vectors and so now we
记得这三个十维字符矢量的融合

recall [ri'kɔ:l]
召回率 一种分类模型 指标，用于回答以下问题：在所有可能的正类别标签中， 模型正确地识别出了多少个？即：
神经网络基础

1:21:04 just want to undo that so this is actually relatively straightforward operation because
只是想撤销这个意思,所以这其实是相对简单的操作,因为

operation
运算 运算
神经网络基础

1:21:10 the backward pass of the what is the view view is just a reprint representation of the array
视图的后向过后方, 视图只是对数组的重印表示

backward pass
反向传播 反向传播
神经网络基础

1:21:15 it's just a logical form of how you interpret the array so let's just reinterpret it to be what it was
这只是你如何解释阵列的逻辑形式 所以让我们重新解释它 成为原来的样子

1:21:20 before so in other words the m is not 32 by 30 it is basically dmcat but if you view it as
换句话说,之前的M不是32比30,3乘30,基本上就是dmcat,但如果你把它看作是

1:21:32 the original shape so just m dot shape you can you can pass some tuples into view
原始形状, 仅是点形, 您可以通过一些图普 进入视图

1:21:40 and so this should just be okay we just re-represent that view and then we
所以,这应该只是没关系 我们只是重新展示这个观点 然后我们

1:21:47 uncomment this line here and hopefully yeah so the derivative of m is correct
取消这条线的内涵 希望是,所以M的衍生物是正确的

derivative
导数 导数
反向传播与梯度

1:21:55 so in this case we just had to re-represent the shape of those derivatives into the original view
所以在这种情况下,我们只需要重新 代表这些衍生物的形状 进入原始观点

1:21:59 so now we are at the final line and the only thing that's left to backpropagate through
现在我们在最后的一线 唯一剩下的东西 来反向宣传

1:22:03 is this um indexing operation here msc at xb so as i did before i copy pasted this line here
是在 xb 的 msc 此处进行平面索引操作, 像我在复制此行之前所做的那样

operation
运算 运算
神经网络基础

1:22:10 and let's look at the shapes of everything that's involved and remind ourselves how this worked
让我们看看所有参与其中的 各种形状,并提醒自己 这如何运作

1:22:15 so m dot shape was 32 by 3 by 10 so it's 32 examples and then we have three characters each
所以点形是32乘3乘3乘10, 也就是32个例子 然后我们各有三个字符

1:22:23 one of them has a 10 dimensional embedding and this was achieved by taking the lookup table c
其中有一个有10维嵌入,这是通过查看表c实现的。

embedding [em'bedɪŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:22:29 which have 27 possible characters each of them 10 dimensional and we looked up um at the rows
上面有27个可能字符 每个有10维的字符 我们抬头看一看

1:22:37 that were specified inside this tensor xb so xb is 32 by 3 and it's basically giving us for each
指定在这个 shor xb 中的 , 要求的 , 以三十二乘三三计 三, 基本上每个都给我们

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:22:44 example the identity or the index of which character is part of that example and so here i'm showing
示例 身份 或 该示例中字符的索引

1:22:52 the first five rows of three of this tensor xb and so we can see that for example here it was
3个高压xb的前5行, 我们可以看到,例如这里

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:22:59 the first example in this batch is that the first character and the first character and the fourth
本批中第一个例子例一是第一个字符和第一个字符,第四个是第四个

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:23:03 character comes into the neural net and then we want to predict the next character in a sequence
字符进入神经网 然后我们想按顺序预测下一个字符

neural net
神经网络 神经网络
神经网络基础

1:23:09 after the character is 114 so basically what's happening here is there are integers inside xb
字符114之后,字符114。 所以这里发生的情况基本上是 xb 内有整数

1:23:17 and each one of these integers is specifying which row of c we want to pluck out right and
每个整数都指定了 哪些 c 行,我们想右转

1:23:24 then we arrange those rows that we've plucked out into 32 by 3 by 10 tensor and we just
然后我们安排那些排 我们从3乘10 10 10

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:23:29 package them in we just package them into this tensor and now what's happening is that we have dmp
把它们包装在我们的容器里, 我们只是把它们包装到这个发压中 现在发生的情况是,我们有

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:23:36 so for every one of these basically plucked out rows we have their gradients now but they're arranged
所以对于每一个这些基本上 都拔出排行 我们现在有了他们的梯度 但是他们已经安排好了

1:23:43 inside this 32 by 3 by 10 tensor so all we have to do now is we just need to route this gradient
3x3x10 10万个阵列,我们现在要做的就是... ...我们只需要绕过这个梯度

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:23:50 backwards through this assignment so we need to find which row of c that every one of these
倒着走过此任务, 所以我们需要找到哪些 c 行

1:23:55 10 dimensional embeddings come from and then we need to deposit them into dc
10维嵌入器来自10维的嵌入器, 然后我们需要把它们存到 DC

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:24:03 so we just need to undo the indexing and of course if any of these rows of c was used multiple times
所以我们只需要取消索引, 当然如果这些 c 行中的任何一行被多次使用

1:24:09 which almost certainly is the case like the row one and one was used multiple times
几乎肯定是这样的,就像第一排和第一列被多次使用

1:24:13 then we have to remember that the gradients that arrive there have to add
然后我们必须记住,到达那里时的梯度必须加上

1:24:18 so for each occurrence we have to have an addition so let's now write this out
所以每次发生的事情都要加一个,所以现在写出来

1:24:22 and I don't actually know of like a much better way to do this than a for loop unfortunately in
我其实不知道 做这个比做一个圆环更好的方法

1:24:26 python so maybe someone can come up with a factorized efficient operation but for now let's just use
皮松,这样也许有人可以提出 一个因素化的有效操作 但是现在我们只用

operation
运算 运算
神经网络基础

1:24:32 for loops so let me create torched out zeros like c to initialize just 27 by 10 tensor of all zeros
以启动光速为27x10度的零点

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:24:43 and then honestly for k in range xb.shape at zero maybe someone has a better way to do this
并且诚实地 k 在 xb. 形状范围为零, 以 K 的 范围为 xb.shape 零位, 也许有人有更好的方法做到这一点

1:24:50 but for j in range xb.shape at one this is going to iterate over all the all the elements of xb
但对于 j , 在范围 xb. 形状 上的一个位置, 这将对 xb 的所有元素进行循环

1:25:01 all these integers and then let's get the index at this position so the index is basically xb at
所有的这些整数, 然后让我们在这个位置获得索引, 所以索引基本上在

1:25:09 kj so that an example of that like is 11 or 14 and so on and now in a forward pass we took
那样的例子就11或14,等等。 现在,我们通过一个前传

forward pass
前向传播 前向传播
神经网络基础

1:25:19 we basically took the row of c at index and we deposited it into m at kj that's what happened
我们基本上在指数上 取了C排的C, 然后我们把它存到KJ的M

1:25:31 that's where they are packaged so now we need to go backwards and we just need to route
在那里他们包装 所以我们现在需要往后走 我们只需要路线

1:25:36 dm at the position kj we now have these derivatives for each position and it's 10 dimensional
位置为 kj 的 dm , 我们现在有这些衍生物 用于每个位置, 并且是10维

1:25:45 and you just need to go into the correct row of c so dc rather at ix is this but plus equals
并且你只需要进入正确的 c行 所以dc 而不是在ix

标签 · 表达式 1:25:55

1:25:55 because there could be multiple occurrences like the same row could have been used many many times
因为可能有很多像同一个行一样的多发生事件 可能已经多次使用

1:26:00 and so all of those derivatives will just go backwards through the indexing and they will add
所以所有这些衍生物 都会从索引的后退 并添加

1:26:06 so this is my candidate solution let's copy it here let's uncomment this and cross our fingers
所以这是我的候选方案 来复制一下 我们不要提这个,然后交叉

1:26:20 yay so that's it we've back propagated through this entire beast so there we go totally made sense
整个野兽的传播 我们完全理解

1:26:30 so now we come to exercise two it basically turns out that in this first exercise we were
现在我们来练习两次, 基本上证明 在第一次练习中

1:26:35 doing way too much work we were back propagating way too much and it was all good practice and so on
做太多工作,我们回来宣传太多,这是所有的良好做法等等

1:26:40 but it's not what you would do in practice and the reason for that is for example here I separated out
但实际上你不会做什么 原因就在这里 我分开了

1:26:46 this loss calculation over multiple lines and I broke it up all all to like its smallest atomic
计算多线损失计算 我把它拆散了 喜欢它最小的原子

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:26:51 pieces and we back propagated through all of those individually but it turns out that if you
我们又在众人中各自散布我们所记录的消息,如果你们

1:26:56 just look at the mathematical expression for the loss then actually you can do the differentiation
只要看看损失的数学表达式, 然后实际上你可以做区别

expression
表达式 表达式
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:27:02 on pen and paper and a lot of terms cancel and simplify and the mathematical expression you
笔纸上和纸上 以及很多术语 取消和简化,数学表达式

expression
表达式 表达式
神经网络基础

1:27:07 end up with can be significantly shorter and easier to implement than back propagating through all
the
其最终可以大大缩短,执行起来比在各方面进行反向宣传要容易得多。

1:27:11 little pieces of everything you've done so before we had this complicated forward pass
之前你所做的一切的细小碎片 之前我们有这个复杂的前进通道

forward pass
前向传播 前向传播
神经网络基础

1:27:16 going from logits to the loss but in pytorch everything can just be glued together into a
从登入到丢失 但用火炉把所有的东西粘在一起

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:27:21 single call at that cross entropy you just pass in logits and the labels and you get the
单一呼叫,在那个十字环的 entropy 你刚刚通过登录和标签

cross entropy
交叉熵 交叉熵
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:27:26 exact same loss as I verify here so our previous loss and the fast loss coming from the
损失与我在此核实的损失完全相同 因此我们之前的损失和快速损失

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:27:31 chunk of operations as a single mathematical expression is the same but it's much much faster in
操作的块块作为单一数学表达式是相同的, 但速度要快得多

expression
表达式 表达式
神经网络基础

1:27:37 a forward pass it's also much much faster in backward pass and the reason for that is if you
后转速度要快得多 原因是如果你

forward pass
前向传播 前向传播
神经网络基础

backward pass
反向传播 反向传播
神经网络基础

1:27:42 just look at the mathematical form of this and differentiate again you will end up with a
只需看看数学形式的这个 并再次区分 你最终会有一个

1:27:45 very small and short expression so that's what we want to do here we want to in a single
很小,很短的表达方式 所以我们想在这里做的就是

expression
表达式 表达式
神经网络基础

1:27:50 operation or in a single go or like very quickly go directly into the logits and we need to
或在一个单行中操作, 或者在一个运行中, 或者类似一样非常快速地直接进入登录, 我们需要

operation
运算 运算
神经网络基础

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:27:57 implement the logits as a function of logits and yb's but it will be significantly shorter than
执行日志作为日志和 Yb 的函数功能,但将大大短于

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:28:04 whatever we did here where to get to the logits we had to go all the way here so all of this work
不管我们在这里做什么 哪里去登入登机记录 我们不得不一路走到这里 所以所有的工作

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:28:10 can be skipped in a much much simpler mathematical expression that you can implement here so you
可以在简单得多的数学表达式中跳过, 您可以在此执行此表达式, 这样您就可以

expression
表达式 表达式
神经网络基础

1:28:17 can give it a shot yourself basically look at what exactly is the mathematical expression of loss
自己可以给它一个镜头 基本上看看什么是损失的数学表达

expression
表达式 表达式
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:28:23 and differentiate with respect to the logits so let me show you a hint you can of course try it
区分记录记录,让我给你们看一个提示,你当然可以试试

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:28:29 fully yourself but if not I can give you some hint of how to get started mathematically so basically
完全的你自己,但如果我不行 我可以给你一些提示 如何开始数学学的

1:28:37 what's happening here is we have logits then there's a softmax that takes the logits and gives
发生的情况是 我们有登录记录 然后有一个软式的 自动取出登录记录并给

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:28:41 you probabilities then we are using the identity of the correct next character to pluck out a row
您的概率, 然后我们使用正确的下一个字符的身份来拔出列

1:28:48 of probabilities take the negative log of it to get our negative log probability and then we
概率的概率 将负日志的负日志 以获得负日志概率 然后我们

probability
概率 概率
数学与微积分

1:28:53 average up all the log probabilities or negative log probabilities to get our loss
平均增加所有日志概率或负日志概率以获得损失

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:28:59 so basically what we have is for a single individual example rather we have that loss is equal to
所以基本上我们所拥有 是一个单个例子,而不是我们有

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:29:05 negative log probability where p here is kind of like thought of as a vector of all the probabilities
负日志概率, p在这里的p 是一种相似的想象, 被视作所有概率的矢量

probability
概率 概率
数学与微积分

vector
向量 向量
数学与微积分

1:29:12 so at the yth position where y is the label and we have that p here of course is the
的 Y 是标签,我们有这个P 当然是

label ['leibl]
标签 在监督式学习中，标签指样本 的 " 答案 " 或 " 结果 " 部分。有标签数据集中的每个样本 都包含一个或多个特征以及一个标签。例如，在房屋数据集中，特征可能包括卧室数、卫生 间数以及房龄，而标签则可能是房价。在垃圾邮件检测数据集中，特征可能包括主题行、发件人以及电子邮件本身，而标签则可能是 " 垃圾邮件 " 或 " 非垃圾邮件 "
神经网络基础

1:29:19 softmax so the ith component of p of this probability vector is just the softmax function
软最大,所以此概率矢量 p 的 IH 组件只是软最大函数

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

probability
概率 概率
数学与微积分

1:29:26 so raising all the logits basically to the power of e and normalizing so everything sums to one
使所有登录都基本上 向电子的力量和正常化的动力 使得一切变成一个

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:29:35 now if you write out p of y here you can just write out the softmax and then basically what
如果你现在写在这里的p y 你可以写在软式的y,然后基本上什么

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:29:40 we're interested in is we're interested in the derivative of the loss with respect to the ith
我们感兴趣的是,我们有兴趣 损失的衍生物 与它有关h

derivative
导数 导数
反向传播与梯度

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:29:45 logit and so basically it's a d by d li of this expression here where we have l indexed with the
logit 所以基本上它是一个 d by d li 的表达式, 这里我们用

expression
表达式 表达式
神经网络基础

1:29:54 specific label y and on the bottom we have a sum over j of e to the lj and the negative log of all that
具体标签y和底部的标签,我们有一个在j的 j 上积的 e 到 lj 和所有这些的负日志

label ['leibl]
标签 在监督式学习中，标签指样本 的 " 答案 " 或 " 结果 " 部分。有标签数据集中的每个样本 都包含一个或多个特征以及一个标签。例如，在房屋数据集中，特征可能包括卧室数、卫生 间数以及房龄，而标签则可能是房价。在垃圾邮件检测数据集中，特征可能包括主题行、发件人以及电子邮件本身，而标签则可能是 " 垃圾邮件 " 或 " 非垃圾邮件 "
神经网络基础

1:30:00 so potentially give it a shot pen and paper and see if you can actually derive the expression
来试试看能不能真正产生表达式

expression
表达式 表达式
神经网络基础

1:30:04 for the loss by d li and then we're going to implement it here okay so i'm going to give away
损失的Dli 然后我们在这里执行 好,所以我要放弃

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:30:10 the result here so this is some of the math i did to derive the gradients analytically and so we see
由此得出了这里的结果,所以这是一些数学, 我做了一些数学,通过分析来计算梯度, 所以我们可以看到

1:30:17 here that i'm just applying the rules of calculus from your first or second year of bachelor's degree
这里我就是在应用 从你学士学位一或二年级开始的微积分规则

1:30:21 if you took it and we see that the expression is actually simplified quite a bit you have to separate
如果你拿了它,我们可以看到这个表达式实际上简化了 相当一点点,你必须分开

expression
表达式 表达式
神经网络基础

1:30:26 out the analysis in the case where the ith index that you're interested in inside logits is
分析出来 在情况下, 当你感兴趣的Ish指数 对内部登录是

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:30:31 either equal to the label or it's not equal to the label and then the expression is simplified
等于标签或不等于标签,然后表达式简化

label ['leibl]
标签 在监督式学习中，标签指样本 的 " 答案 " 或 " 结果 " 部分。有标签数据集中的每个样本 都包含一个或多个特征以及一个标签。例如，在房屋数据集中，特征可能包括卧室数、卫生 间数以及房龄，而标签则可能是房价。在垃圾邮件检测数据集中，特征可能包括主题行、发件人以及电子邮件本身，而标签则可能是 " 垃圾邮件 " 或 " 非垃圾邮件 "
神经网络基础

expression
表达式 表达式
神经网络基础

1:30:36 and canceled in a slightly different way and what we end up with is something very very simple
以稍有不同的方式取消 我们最后的结局很简单

1:30:41 if we either end up with basically p and i where p is again this vector of probabilities after a soft
如果我们要么最终 基本上p和i 与p和i是相同的,p又是一个 柔软的概率矢量

vector
向量 向量
数学与微积分

1:30:47 max or p and i minus one where we just simply subtract to one but in any case we just need
最大或p和i减去1, 我们只要简单地减一个, 但无论如何我们只需要

1:30:53 to calculate the softmax p and then in the correct dimension we need to subtract to one
来计算软顶p,然后在正确的尺寸中,我们需要将一个减到

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

dimension
维度 维度
数学与微积分

1:30:59 and that's the gradient the form that it takes analytically um so let's implement this basically
并且这就是梯度 它的分析形式,嗯。 所以让我们基本上执行这个

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:31:04 and we have to keep in mind that this is only done for a single example but here we are working with
我们必须牢记,这样做只是为了一个例子,但我们现在正与

1:31:08 batches of examples so we have to be careful of that and then the loss for a batch is the average
举例的批次,因此我们必须小心这一点,然后,一批损失是平均损失。

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:31:14 loss over all the examples so in other words is the example for all the individual examples
在所有例子中,所有损失的示例,换句话说,就是所有个别例子的例子。

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:31:19 is the loss for each individual example summed up and then divided by n and we have to back
是一个单个例子的损失,然后以n除以n来汇总,我们必须返回

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:31:24 propagate through that as well and be careful with it so d logits is going to be f dot softmax
透过这个传播,要小心注意 登入的就是点软麦

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:31:30 uh pytorch has a softmax function that you can call and we want to apply the softmax on the logits
iuhpytoch 具有软max 功能, 您可以调用, 我们想要在登录上应用软max

pytorch
PyTorch PyTorch
代码与工程

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:31:38 and we want to go in the dimension that is one so basically we want to do the softmax along the
我们想要进入一个维度, 也就是我们想要沿着这个维度 做软式的

dimension
维度 维度
数学与微积分

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:31:44 rows of these logits then at the correct positions we need to subtract a one so d logits at iterating
这些对帐行的行数,然后在正确位置上,我们需要在迭代时减去一个如此相偏的登入

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:31:52 over all the rows and indexing into the columns provided by the correct labels inside yb
在所有行中,并用 Yb 内正确的标签提供的列索引

1:32:00 we need to subtract one and then finally it's the average loss that is the loss and in the
最终损失是平均损失 损失和

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:32:06 average there's a one over n of all the losses added up and so we need to also back propagate
平均损失额有超过一亿,所以我们还需要回传

1:32:12 through that division so the gradient has to be scaled down by by n as well because of the mean
以通过这种除法来使梯度缩小n,因为平均

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

概率 · 缩放 1:32:20

1:32:20 but this otherwise should be the result so now if we verify this we see that we don't get an
但要不然结果应该是这个,所以现在如果我们核实这一点,我们就会发现我们没有得到

1:32:26 exact match but at the same time the maximum difference from logits from pytorch and our d
与 pytorch 和 our d 的最大日志差异

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

pytorch
PyTorch PyTorch
代码与工程

1:32:33 logits here is on the order of 5e negative 9 so it's a tiny tiny number so because of
此处的登录在5 e 负负 9 的顺序下 所以它是一个小数,所以因为

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:32:40 floating point wonkiness we don't get the exact bitwise result but we basically get the correct
浮点圆基度 我们没有得到准确的比位结果 但我们基本上得到了正确的

1:32:45 answer approximately now i'd like to pause here briefly before we move on to the next exercise
回答大致现在,我想在这里简短地暂停一下,然后我们接着进行下一次演习。

1:32:52 because i'd like us to get an intuitive sense of what d logits is because it has a beautiful and
因为我想让我们直觉地了解 登入的是什么 因为它有一个美丽的

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:32:57 very simple explanation honestly so here i'm taking the logits and i'm visualizing it and we can see
简单解释,所以我在这里 使用登录 我想象它,我们可以看到

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:33:04 that we have a batch of 32 examples of 27 characters and what is the logits intuitively right? d
上面有32个27个字符的样本, 直觉上记录对吗? d

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:33:11 logits is the probabilities that the probabilities matrix in the forward pass but then here these
逻辑是前方通道概率矩阵的概率, 但在此显示这些

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

matrix
矩阵 矩阵
数学与微积分

1:33:16 black squares are the positions of the correct indices where we subtracted a 1 and so what is
黑方是正确指数的位置,我们在此减去 1,所以什么是

1:33:23 this doing right? these are the derivatives on d logits and so let's look at just the first row
这样做对吗? 这些是D登录上的衍生物,所以让我们看看第一排

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:33:30 here so that's what i'm doing here i'm calculating the probabilities of these logits and that i'm
所以我在这里做,我正在计算 这些登录的概率

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:33:35 taking just the first row and this is the probability row and then the logits of the first row
仅取第一行,这是第一行的概率行和第一行的登录

probability
概率 概率
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:33:41 and multiplying by n just for us so that we don't have the scaling by an in here and everything is
乘以n,我们只是这样 我们没有规模的在这里

scaling ['skeiliŋ]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

1:33:46 more interpretable we see that it's exactly equal to the probability of course but then the position
更可以解释的,我们发现它与概率完全相等 当然,但然后位置

probability
概率 概率
数学与微积分

1:33:52 of the correct index has a minus equals one so minus one on that position and so notice that
键的正对索引在该位置上有一个减等于1,因此该索引的负数等于1,所以通知

1:33:59 if you take d logits at zero and you sum it it actually sums to zero and so you should think
如果你在零点时将D登录数取为零,然后将其计算成等于零,所以你应该想

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:34:06 of these gradients here at each cell as like a force we are going to be basically pulling down
每一个单元格的这些梯度 就像一种力量 我们基本上要往下拉

1:34:15 on the probabilities of the incorrect characters and we're going to be pulling up on the probability
错误字符概率的概率 我们将加快概率

probability
概率 概率
数学与微积分

1:34:21 at the correct index and that's what's basically happening in each row and the the amount of push
正确的指数,这就是每行发生的情况 以及推力的数量

1:34:30 and pull is exactly equalized because the sum is zero so the amount to which we pulled down
并且拉动完全对齐, 因为总和是零 所以我们拉下来的金额

1:34:35 on the probabilities and the amount that we push up on the probability of the correct character is equal
概率和我们推高的正确字符概率值相等

probability
概率 概率
数学与微积分

1:34:41 so the repulsion and the attraction are equal and think of the neural net now as a
所以反感和吸引力是平等的 并且把神经网现在想成

neural net
神经网络 神经网络
神经网络基础

1:34:46 like a massive pulley system or something like that we're up here on top of the logits and
像是巨大的滑轮系统之类的 我们在上面的登机记录上

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:34:52 we're pulling up we're pulling down the probabilities of incorrect and pulling up the
我们拉起我们拉起我们 拉下错误概率 并拉起

1:34:55 probability of the correct and in this complicated pulley system because everything is mathematically
正确概率和在这个复杂的滑轮系统中的概率,因为从数学学上说,一切都是

probability
概率 概率
数学与微积分

1:35:00 just determined just think of it as sort of like this tension translating to this complicating
仅仅确定一下,只是把它想成 就像这种紧张的紧张 变成这样复杂的

1:35:05 pulling mechanism and then eventually we get a tug on the weights and the biases and basically in
最终我们得到拖拉机 最后我们得到一个拖拉 拉在重量和偏向上

1:35:11 each update we just kind of like tug in the direction that we'd like for each of these elements
每一个更新 我们只是像拖 方向,我们希望 对于其中的每一个元素

1:35:15 and the parameters are slowly given in to the tug and that's what training in neural net
参数慢慢地被注入拖网 这就是神经网的训练

training ['treinin]
训练 确定构成模型的理想参数 的过程。
神经网络基础

neural net
神经网络 神经网络
神经网络基础

1:35:20 kind of like looks like on a high level and so I think the the forces of push and pull in
像是高层次的高度 所以我想推力

1:35:25 these gradients are actually very intuitive here we're pushing and pulling on the correct answer
这些梯度其实非常直观 我们推拉正确的答案

1:35:30 and the incorrect answers and the amount of force that we're applying is actually proportional to
和不正确的答案 和我们所应用的武力数量 实际上与

1:35:36 the probabilities that came out in the forward pass and so for example if our probabilities came out
前面的概率,比如说,如果我们的概率出来了

forward pass
前向传播 前向传播
神经网络基础

1:35:41 exactly correct so they would have had zero everywhere except for one at the correct position
准确无误,因此除位置正确者外,无处是零

1:35:47 then the the logits would be all a row of zeros for that example there would be no push and pull
那样的话,对帐单就成了一排零数的行, 就没有推拉

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:35:53 so the amount to which your prediction is incorrect is exactly the amount by which
所以,你的预测不正确的数额 正好是

1:35:58 you're going to get a pull or push in that dimension so if you have for example a very
就会拉拉或推进那个维度 所以如果你有

dimension
维度 维度
数学与微积分

1:36:03 confidently mispredicted element here then what's going to happen is that element is going to be
充满自信的错误预设元素 在这里,然后会发生什么 将发生的事情是 这个元素将是

1:36:09 pulled down very heavily and the correct answer is going to be pulled up to the same amount
大量地拖下水 正确的答案将拉到相同数量

1:36:15 and the other characters are not going to be influenced too much so the amount to which
其它字符不会受到太大的影响,

1:36:20 you mispredict is then proportional to the strength of the pull and that's happening
你误解了 与拉力成比例 然后发生

批归一化 · 批次大小 1:36:25

1:36:25 independently in all the dimensions of this of this tensor and it's sort of very intuitive
独立地在这种抗拉的所有维度上 都存在,而且它非常直观

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

1:36:30 and very easy to think through and that's basically the magic of the cross entropy loss and what it's
并且很容易思考,基本上这就是 十字环环球丧失的魔法

cross entropy
交叉熵 交叉熵
数学与微积分

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:36:34 doing dynamically in the backward pass of the neural net so now we get to exercise number three
动态地在神经网的后脑网 后脑关下 进行动态的演练 所以我们现在开始练习第三号

backward pass
反向传播 反向传播
神经网络基础

neural net
神经网络 神经网络
神经网络基础

1:36:40 which is a very fun exercise depending on your definition of fun and we are going to do for
这是一种非常有趣的练习,取决于你对乐趣的定义 我们将为

1:36:45 batch normalization exactly what we did for cross entropy loss in exercise number two
第二期演习中 分批处理的原样 完全正常化

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

cross entropy
交叉熵 交叉熵
数学与微积分

1:36:49 that is we are going to consider it as a glued single mathematical expression
即我们将会把它视为 一个粘结的单数学表达式

expression
表达式 表达式
神经网络基础

1:36:53 and back propagate through it in a very efficient manner because we are going to derive a much
并透过它以非常高效的方式传播,因为我们将获得很多

1:36:57 simpler formula for the backward pass of batch normalization and we're going to do that using
分批正常化后端路口的简单公式

backward pass
反向传播 反向传播
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:37:02 pen and paper so previously we've broken up batch normalization into all of the little intermediate
之前我们把批量正常化 打破了所有中间体

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:37:07 pieces and all the atomic operations inside it and then we back propagate it through it
和所有原子操作的碎片 然后我们又通过它来传播

1:37:11 one by one now we just have a single sort of forward pass of a batch term and it's all
现在我们有一个单一个的 分批连期的远期传承

forward pass
前向传播 前向传播
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:37:18 glued together and we see that we get the exact same result as before now for the batch backward
粘合在一起,我们看到我们得到的结果与以前一样 对于批次后退

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:37:24 pass we'd like to also implement a single formula basically for back propagating through this entire
也执行一个单一的公式 基本上用来回传整个过程

1:37:29 operation that is the batch normalization so in the forward pass previously we took
即批量正常化, 之前我们用过的前传

operation
运算 运算
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:37:34 h p b n the hidden states of the pre batch normalization and created h preact which is
h p b n 批次正常化前的隐藏状态,并创造了hp 预言,即

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:37:41 the hidden states just before the activation in the batch normalization paper h p b n is x
批量正常化纸中激活前的隐藏状态 h p b n is x

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:37:47 and h preact is y so in the backward pass what we'd like to do now is we have d h preact and we'd
后过道的hpact是y 我们现在想做的是 我们有dhpact,我们愿意

backward pass
反向传播 反向传播
神经网络基础

1:37:53 like to produce d h previan and we'd like to do that in a very efficient manner so that's the name
想要生产dhh 预产禽 我们想以非常高效的方式这样做 所以这是名字

1:37:59 of the game calculate d h previan given d h preact and for the purposes of this exercise we're going
计算 d h 之前的 时间,并为了本次练习的目的

1:38:06 to ignore gamma and beta and their derivatives because they take on a very simple form in a
忽略伽马和贝塔及其衍生物,因为它们在

1:38:11 very similar way to what we did up above so let's calculate this given that right here so to help
和我们在上面所做的非常相似 让我们计算一下这个,在这里,这样可以帮助

1:38:19 you a little bit like I did before I started off the implementation here on pen and paper and I took
你跟我以前一样 开始用笔纸做成

implementation
实现 实现
代码与工程

1:38:26 two sheets of paper to derive the mathematical formulas for the backward pass and basically
用于为后向通道和基本

backward pass
反向传播 反向传播
神经网络基础

1:38:31 to set up the problem just write out the mu sigma square variance x i hat and y i exactly
设置问题, 只要写出 mu sigma 方差 x i hat 和 y i 即可

1:38:38 as in the paper except for the best of correction and then in the backward pass we have the derivative
和纸上一样 除了最好的校正 和后传的

backward pass
反向传播 反向传播
神经网络基础

derivative
导数 导数
反向传播与梯度

1:38:45 of the loss with respect to all the elements of y and remember that y is a vector there's
y 中所有元素的损失,并记住y是矢量

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

vector
向量 向量
数学与微积分

1:38:49 there's multiple numbers here so we have all the derivatives with respect to all the y's
这里有很多数字 所以我们有所有y的衍生物

1:38:56 and then if there's a gamma and a beta and this is kind of like the compute graph
如果有伽马和贝塔,这就像计算图

1:39:01 the gamma and beta there's the x hat and then the mu and the sigma square and the x
和BET的X帽子 然后是MU,Sigma广场和x

1:39:07 so we have d l by d y i and we want d l by d x i for all the i's in these vectors
所以,我们有 dl by d y i, 我们需要d l by d x i 用于这些矢量中的所有 I

1:39:15 so this is the compute graph and you have to be careful because I'm trying to note here that
所以这是计算图,你必须小心 因为我想在这里指出

1:39:21 these are vectors there's many nodes here inside x x hat and y but mu and sigma sorry sigma square
它们是矢量,这里有很多节点 在 xx 帽子和y的内部,但 mu和sigma 对不起 sigma

1:39:29 are just individual scalars single numbers so you have to be careful with that you have
仅仅是单量的单量标量,所以你必须小心

1:39:33 to imagine there's multiple nodes here or you're going to get your math wrong
想象这里有多节点 或者你数学错误

1:39:37 so as an example I would suggest that you go in the following order one two three four in terms
举个例子,我建议你按以下顺序排列

1:39:44 of the back propagation so back propagating to x hat then to sigma square then into mu and then into x
后背传播, 后背向后传播到x 帽子,然后向西格玛方形, 然后进入mu,然后进入x

back propagation
反向传播 反向传播
神经网络基础

1:39:52 just like an entopological sort in micrograd we would go from right to left you're doing the
就像微研究生的昆虫学类 我们从右到左,你正在做

micrograd
micrograd 自动微分库 micrograd 自动微分库
神经网络基础

1:39:56 exact same thing except you're doing it with symbols and on a piece of paper so for number one
完全一样 除了你用符号做 纸上写一个符号

1:40:03 I'm not giving away too much if you want d l of d x i hat then we just take d l by d y i and multiply
如果你想要Dl的Dxl 帽子,我不会放弃太多。 然后我们只要拿dl by d y i 乘

1:40:12 by gamma because of this expression here where any individual y i is just gamma times x i hat
yi只是伽马乘以I hat

expression
表达式 表达式
神经网络基础

1:40:19 plus beta so it didn't help you too much there but this gives you basically the derivatives
加上贝贝塔,所以对你没有太大帮助 但这基本上就给了你衍生物

1:40:25 for all the x hats and so now try to go through this computational graph and derive what is d l by d
尝试通过这个计算图 得出什么是 d l 乘 d

1:40:33 sigma square and then what is d l by d mu and then what is d l by d x eventually so give it a go and
以 d 乘 d mu 和 d 乘 乘 d x 的 等号 方形, 然后是 d 乘 D 的 d x , 所以最终试一试

1:40:41 i'm going to be revealing the answer one piece at a time okay so to get d l by d sigma square
我将展示一个答案 一次一个回答一块 好,这样得到Dl

1:40:47 we have to remember again like I mentioned that there are many x's x hats here and remember
我们必须再次记住,就像我提到过的那样 这里有很多x的x帽子 记住

1:40:53 that sigma square is just a single individual number here so when we look at the expression
也就是这个Sigma广场 就是这里的单数 所以当我们看这个表达式

expression
表达式 表达式
神经网络基础

1:40:59 for d l by d sigma square we have that we have to actually consider all the possible paths that
德西格玛广场 我们不得不实际考虑

1:41:08 we basically have that there's many x hats and they all feed off from they all depend on sigma
基本上我们有 很多x帽子,它们都靠 希格玛来取食

1:41:13 square so sigma square has a large fan out there's lots of arrows coming out from sigma square
方形西格玛广场有大扇扇风 有很多箭头从西格玛方形飞出来

1:41:18 into all the x hats and then there's a back propagating signal from each x hat into sigma square
进入所有的x帽子 然后有一个背后传播信号 从每x帽子到西格玛方形

1:41:25 and that's why we actually need to sum over all those uh eyes from i equal to 1 to m of the d l by d
所以我们需要把这些双眼的 光量从我等于1到1米

1:41:33 x i hat which is the global gradient times the x i hat by d sigma square which is the local gradient
x 顶,即全球梯度乘以x 顶乘以 d sigma 方形,即本地梯度

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:41:42 of this operation here and then mathematically I'm just working it out here and I'm simplifying
和这个操作的数学上,然后我在这里工作,我简化

operation
运算 运算
神经网络基础

1:41:47 and you get a certain expression for d l by d sigma square and we're going to be using this
并且你得到一个dl的表达式,D by d sigma广场 我们将会使用这个

expression
表达式 表达式
神经网络基础

1:41:52 expression when we back propagate into mu and then eventually into x so now let's continue our back
当我们回到变形后, 最终再升为x, 然后我们又回到x, 表达时的表情。 所以现在让我们继续我们的背部

expression
表达式 表达式
神经网络基础

1:41:57 propagation into mu so what is d l by d mu now again be careful that mu influences x hat and x hat
以Mus的传播方式, dl by d mu 现在再次注意: mus 影响x hat 和 x hat

1:42:05 is actually lots of values so for example if our mini batch size is 32 as it is in our example
实际上是很多的值,例如,如果我们的小批量大小是32,就像我们的例子一样

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:42:10 that we were working on then this is 32 numbers and 32 arrows going back to mu and then mu
那时我们正在研究的,这是32个数字和32支箭 返回到穆,然后是穆

1:42:16 going to sigma square is just a single arrow because sigma square is a scalar so in total there are 33
将到 sigma 方块是一个箭头, 因为 Sigma 广场是一块标语, 所以总共33支

scalar
标量 标量
数学与微积分

1:42:21 arrows emanating from you and then all of them have gradients coming into mu and they all need
箭从你发出的,然后,他们都有梯度,它们都结成木质,它们都需要

1:42:27 to be summed up and so that's why when we look at the expression for d l by d mu I am summing up
所以当我们看D的表达方式时 我总结

expression
表达式 表达式
神经网络基础

1:42:34 over all the gradients of d l by d x i hat times d x i hat by d mu so that's the that's this

1:42:41 arrow and that's 32 arrows here and then plus the one arrow from here which is d l by d sigma square
箭头,这是32支箭头 这里加上一个箭头,就是dl by d sigma 方形的 d l

1:42:48 times d sigma square by d mu so now we have to work out that expression and let me just reveal the
以d sigma 平方乘以 d gum 平方 和 d m mu 。 所以我们现在必须研究出这个表达方式, 让我先透露

expression
表达式 表达式
神经网络基础

1:42:53 rest of it simplifying here is not complicated the first term and you just get an expression here
简化这里的其余部分 并不复杂第一学期 你在这里只用表达

expression
表达式 表达式
神经网络基础

1:43:01 for the second term though there's something really interesting that happens
第二次连任 虽然有一件有趣的事情发生

1:43:04 when we look at the sigma square by d mu and we simplify at one point if we assume
当我们用d mu 来看待Sigma方形时, 如果我们假设

1:43:11 that in a special case where mu is actually the average of x i's as it is in this case then
在一个特殊情况下,如果mu是实际的平均值 x l 和本案一样

1:43:18 if we plug that in then actually the gradient vanishes and becomes exactly zero and that makes
如果我们插入了那个插头, 那么梯度实际上消失 并且变成零, 从而

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:43:24 the entire second term cancel and so these if you just have a mathematical expression like this
第二个任期的第二个整个任期取消, 所以如果您只有像这样数学表达式的话, 将取消这些

expression
表达式 表达式
神经网络基础

1:43:30 and you look at d sigma square by d mu you would get some mathematical formula for how mu
并且你看看 d sigma 方形的 d mu , 你会得到一些数学公式 如何 mu

1:43:36 impacts sigma square but if it is the special case that mu is actually equal to the average
以正方形表示,但在特殊情况下, 粘合物实际上等于平均值

1:43:41 as it is in the case of pastoralization that gradient will actually vanish and become zero
而在放牧的情况下,梯度实际上将消失,变成零

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:43:46 so the whole term cancels and we just get a fairly straightforward expression here for d l by d
所以整个术语取消, 我们在这里得到一个相当直截的表达方式 表示dl by d

expression
表达式 表达式
神经网络基础

1:43:51 mu okay and now we get to the craziest part which is deriving d l by d x i which is ultimately
现在我们到达最疯狂的部分 产生 dl by d x i,最终

1:43:57 what we're after now let's count first of all how many numbers are there inside x as I mentioned
我们接下来要做什么,先算算一下 有多少数字在我提到的x内部

1:44:04 there are 32 numbers there are 32 little x i's and let's count the number of arrows emanating from each
里面有32个数字,32个小的xl's i's 和让我们计算一下每个箭头的箭数

1:44:09 x i there's an arrow going to mu an arrow going to sigma square and then there's an arrow going to x
箭头向西格玛方形方向方向,然后箭头向x方向方向方向

1:44:16 hat but this arrow here let's group now that a little bit each x i hat is just a function of x i
但这个箭头在这里,让我们来组一下, 现在每只一顶一顶一吨一吨一升帽子 只需要一吨一公尺的函数

1:44:23 and all the other scalars so x i hat only depends on x i and none of the other x's and so
所有其他的卡勒,所以... ...我帽子只取决于xi 而其他的没有,等等

1:44:30 therefore there are actually in this single arrow there are 32 arrows but those 32 arrows are going
而这32支箭头正在前进

1:44:36 exactly parallel they don't interfere they're just going parallel between x and x hat you can look at
完全平行,它们不会干扰 它们只是与X和x帽子平行 你可以看

1:44:41 it that way and so how many arrows are emanating from each x i there are three arrows mu sigma
如此一来,每一支x1的箭 有多少箭从每支x1升 有三个箭

1:44:47 square and the associated x hat and so in back propagation we now need to apply the chain rule
和相连的x帽子等 在后传播中 我们现在需要应用链规则

back propagation
反向传播 反向传播
神经网络基础

chain rule
链式法则 链式法则
反向传播与梯度

1:44:54 and we need to add up those three contributions so here's what that looks like if i just write
我们需要加起来这三笔捐款 所以如果我写

1:44:59 that out we have we're going through we're chaining through mu sigma square and through x hat and
我们通过我们正经历的 链条 通过musigma广场和x帽子

1:45:07 those three terms are just here now we already have three of these we have dl by d x i hat we have dl by
这三个条件都在这里 现在我们有三个条件了 我们有Dxi 帽子

1:45:16 d mu which we derived here and we have dl by d sigma square which we derived here but we need
由我们在这里产生,我们从这里得到的 mum 我们从这里得到D sigma方形

1:45:21 three other terms here d this one this one and this one so I invite you to try to derive
三个其他条件 这里,这个条件... ...d 这个这个这个...

1:45:27 them it's not that complicated you're just looking at these expressions here and differentiating with
并不是那么复杂 你只是看这些表达方式

1:45:32 respect to x i so give it a shot but here's the result or at least what I got um yeah I'm just
来试试看,但结果是这样的 至少我得到了什么 嗯,我只是

1:45:43 I'm just differentiating with respect to x i for all of these expressions and honestly I don't think
我只是在区分xi的表达方式 说实话 我不认为

1:45:46 there's anything too tricky here it's basic calculus now what gets a little bit more tricky
这里有什么太难的 现在有基本微积分了 更难处理

1:45:51 is we are now going to plug everything together so all of these terms multiplied with all of these
我们现在要把所有的都塞在一起 所以所有这些条件 都会乘以所有

1:45:56 terms and added up according to this formula and that gets a little bit hairy so what ends up happening is
并且根据这个公式加起来的术语 和按照这个公式,这得到 有点毛毛小 所以最后会发生什么

1:46:05 you get a large expression and the thing to be very careful with here of course is
有个大表情,这里要非常小心的是

expression
表达式 表达式
神经网络基础

1:46:10 we are working with a dl by d xi for specific i here but when we are plugging in some of these
我们在这里用一个dl by dxxx 来工作, 具体为i, 但是当我们插入了其中的一些

1:46:16 terms like say this term here dl by d sigma squared you see how dl by d sigma squared I
字词如此术语, 这里的 dl by d sigma 方形, 您可以看到如何 dl 由 d sigmar 方形

1:46:24 end up with an expression and I'm iterating over little i's here but I can't use i as the
最终我表现的一言一语, 我在这里的一点点在消化 但我不能用我作为

expression
表达式 表达式
神经网络基础

1:46:30 variable when I plug in here because this is a different i from this i this i here is just a
变量时, 我插入此插插此处, 因为它与此i i 不同

1:46:36 placeholder like a local variable for a for loop in here so here when I plug that in you notice
占位符, 就像一个本地变量, 用于循环在此循环, 所以当我插入时, 我在这里通知您

1:46:41 that I renamed the i to a j because I need to make sure that this j is not that this j is not this
我将i改名为j,因为我需要确保这j不是这个j

1:46:47 i this j is like like a little local iterator over 32 terms and so you have to be careful with that
i这个j就像一个小本地的循环器32个条件 所以,你必须小心

1:46:53 when you're plugging in the expressions from here to here you may have to rename i's into j's
当你插入从这里到这里的表达式时, 你可能需要重新命名 i's into J's

1:46:57 and you have to be very careful what is actually an i with respect to the dl by d xi so some of these are
并且你必须非常小心,实际上什么是i 相对于 dxxx的 dl 的 dxx

1:47:05 j's some of these are i's and then we simplify this expression and I guess like the big thing to
j'j的一些这些是 我,然后我们简化这个表达方式 我想像大事情

expression
表达式 表达式
神经网络基础

1:47:13 notice here is a bunch of terms just kind of come out to the front and you can refactor them
注意这里的一连串条款 只是从前面出来 你可以重新解释

1:47:17 there's a sigma squared plus epsilon raised to the power of negative three over two this sigma
以负三比二的负三 推两

1:47:21 square plus epsilon can be actually separated out into three terms each of them are sigma square plus
+ epsilon 和 plus + + 和 epslon 之间可以实际上分离成三个条件,每个条件都是 sigma +

1:47:27 epsilon to the negative one over two so the three of them multiplied is equal to this and then those
以负1乘2等于负1乘二 所以其中三个乘以等于这个 然后是那个

1:47:33 three terms can go different places because of the multiplication so one of them actually comes
三个条件可以因乘数而去不同的地点,所以其中一个条件真的来了

1:47:38 out to the front and will end up here outside one of them joins up with this term and one of
最终会在这里 在一个外的人加入这个任期

1:47:45 them joins up with this other term and then when you simplify the expression you'll notice that
等他们加入到这个术语中 当你简化表达式时 你会注意到

expression
表达式 表达式
神经网络基础

1:47:50 some of these terms that are coming out are just the xi hats so you can simplify just by
其中一些条件正在被公布出来 只是XH帽子 这样你就可以简化

1:47:55 rewriting that and what we end up with at the end is a fairly simple mathematical expression over
here
重写这个,我们最后的结局 是一个很简单的数学表达法

expression
表达式 表达式
神经网络基础

1:48:00 that I cannot simplify further but basically you'll notice that it only uses the stuff we have
我无法进一步简化 但基本上你就会注意到 它只使用我们所拥有的

1:48:06 and it derives the thing we need so we have dl by dy for all the i's and those are used
并且它产生我们需要的东西 所以我们有DL by dy 对于所有我和那些被使用

1:48:13 plenty of times here and also in addition what we're using is these xi hats and xj hats
这里很多次了 还有我们用的还有这些xx 帽子和xj 帽子

1:48:18 and they just come from the forward pass and otherwise this is a simple expression and it gives us
他们从前道而来,或用这个(做)简单的表达,或用它给我们一个

forward pass
前向传播 前向传播
神经网络基础

expression
表达式 表达式
神经网络基础

1:48:24 dl by dx i for all the i's and that's ultimately what we're interested in so that's the end of
最终我们感兴趣的就是这个 所以这就是结束

1:48:30 bachelor backward pass analytically let's now implement this final result okay so I implemented
学士后退通过分析 现在让我们执行这个最后结果 好,我实施

backward pass
反向传播 反向传播
神经网络基础

1:48:37 the expression into a single line of code here and you can see that the max diff is tiny so this
表达式在此以单行代码表示, 您可以看到最大 diff 很小, 所以这个

expression
表达式 表达式
神经网络基础

1:48:43 is the correct implementation of this formula now I'll just basically tell you that getting this formula
正确执行这个公式 现在我基本上告诉你,得到这个公式

implementation
实现 实现
代码与工程

1:48:50 here from this mathematical expression was not trivial and there's a lot going on packed into this one
从这里的数学表达式 并不是微不足道的, 而且有很多事情 包装在这张

expression
表达式 表达式
神经网络基础

1:48:56 formula and this is a whole exercise by itself because you have to consider the fact that this
公式,这是整个过程本身的全过程, 因为你必须考虑这一事实

1:49:01 formula here is just for a single neuron and a batch of 32 examples but what I'm doing here is I'm
这里的公式只是 单神经元和一组32个例子 但我在这里做的是

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:49:07 actually we actually have 64 neurons and so this expression has to in parallel evaluate
实际上我们实际上有64个神经元, 所以这个表达式必须同时评价

expression
表达式 表达式
神经网络基础

1:49:12 the bachelor backward pass for all of those 64 neurons in parallel independently so this has to
独立地平行运行的64个神经元 单身后退通行证 所以这必须

backward pass
反向传播 反向传播
神经网络基础

1:49:17 happen basically in every single column of the inputs here and in addition to that you see how
基本上发生在输入的每个列中 除了你可以看到

1:49:26 there are a bunch of sums here and we need to make sure that when I do those sums that they
这里有一大堆钱 我们需要确保 当我做这些钱的时候

1:49:30 broadcast correctly onto everything else that's here and so getting this expression is just like
正确播放到这里的所有其他东西 所以这个表达方式就像

expression
表达式 表达式
神经网络基础

1:49:35 highly non trivial and I invite you to basically look through it and step through it and it's a
高度非微不足道,我请你基本上通过它来研究,并跨过它。

1:49:38 whole exercise to make sure that this this checks out but once all the shapes agree and once you
整个练习以确保这个检查结果, 但一旦所有形状一致, 一旦你同意

1:49:45 convince yourself that it's correct you can also verify that patrice gets the exact same answer as
也能够证实 Patrice得到的答案与

1:49:49 well and so that gives you a lot of peace of mind that this mathematical formula is correctly
让你能安心地思考 这个数学公式是正确的

1:49:54 implemented here and broadcasted correctly and replicated in parallel for all of the 64 neurons
并同时为所有64个神经元进行正确广播和复制

1:50:01 inside this bathroom layer okay and finally exercise number four asks you to put it all
终于第四运动要求你全放

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:50:05 together and here we have a redefinition of the entire problem so you see that we re-initialize
一起在这里,我们重新定义了整个问题 这样你可以看到,我们重新开始

1:50:11 the neural net from scratch and everything and then here instead of calling the loss that backward
神经网从头到尾 从头到下,然后在这里 而不是把损失称为倒退

neural net
神经网络 神经网络
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:50:16 we want to have the manual back propagation here as we derived it up above so go up copy paste all
我们希望手动手册在这里重新传播 因为我们从上面推出它 所以去复制粘贴

back propagation
反向传播 反向传播
神经网络基础

1:50:22 the chunks of code that we've already derived put them here and derive your own gradients
已经生成的代码块块, 将其放在这里并产生您的梯度

1:50:27 and then optimize this neural net basically using your own gradients all the way to the
然后优化这个神经网 基本上使用你自己的梯度直到

neural net
神经网络 神经网络
神经网络基础

1:50:32 calibration of the batch norm and the evaluation of the loss and I was able to achieve quite a good
批量规范的校准和损失评估,我得以取得相当不错的成就

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

norm
范数 范数
数学与微积分

1:50:36 loss basically the same loss you would achieve before and that shouldn't be surprising because all we've
损失基本上与以前的损失一样 损失应该不会令人惊讶

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:50:41 done is we've really got into loss that backward and we've pulled out all the code and inserted it
做我们真的失去了那个后退的密码 我们把所有代码都拔出来插入了

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:50:47 here but those gradients are identical and everything is identical and the results are
这里的梯度相同,一切完全一样,结果也相同

1:50:51 identical it's just that we have full visibility on exactly what goes on under the hood of
完全相同 只是我们完全能看清

1:50:56 lot that backward in this specific case okay and this is all of our code this is the full backward
和所有我们的代码 这是完全的后退

1:51:02 pass using basically the simplified backward pass for the cross entropy loss and the batch
使用简化的后退通行证处理交叉星体和批次损失

backward pass
反向传播 反向传播
神经网络基础

cross entropy
交叉熵 交叉熵
数学与微积分

1:51:08 normalization so back propagating through cross entropy the second layer the 10h nonlinearity
回到正常状态 通过交叉的 倍增在第二层传播

cross entropy
交叉熵 交叉熵
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:51:15 the batch normalization through the first layer and through the embedding and so you see that
批次正常化 通过第一层和嵌入,所以你看

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:51:20 this is only maybe what is this 20 lines of code or something like that and that's what gives us
这可能是这20行代码的 20行代码之类的东西, 这就是我们得到的东西

1:51:25 gradients and now we can potentially erase loss and backward so the way I have the code set up is
梯度,现在我们有可能消除损失和后退 所以,我设置代码的方式

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:51:31 you should be able to run this entire cell once you fill this in and this will run for only a
您填入此单元格后, 就可以运行整个单元格。 这将只运行一个

1:51:35 hundred iterations and then break and it breaks because it gives you an opportunity to check your
一百次重复,然后打破 它打破,因为它给你一个机会检查你

1:51:40 gradients against pytorch so here our gradients we see are not exactly equal they are
梯度对平板的梯度, 因此在这里我们看到的梯度不完全等同

pytorch
PyTorch PyTorch
代码与工程

训练循环 · PyTorch 1:51:48

1:51:48 approximately equal and the differences are tiny one negative nine or so and I don't exactly know
差数微小一个负9个左右 我不太清楚

1:51:53 where they're coming from to be honest so once we have some confidence that the gradients are
他们的来源是诚实的 一旦我们有信心

1:51:57 basically correct we can take out the gradient checking we can disable this breaking statement
基本上正确,我们可以删除梯度检查, 我们可以禁用此断开语句

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:52:05 and then we can basically disable loss of backward we don't need it anymore
然后我们就可以基本上 停止失去落后的, 我们不再需要它了

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:52:12 feels amazing to say that and then here when we are doing the update we're not going to use
说这句话感觉很惊人,然后在这里 当我们做更新时 我们不会使用

1:52:17 p.grad this is the old way of pytorch we don't have that anymore because we're not doing backward
p. grad 这是旧的烧火方法 我们没有了,因为我们没有落后

pytorch
PyTorch PyTorch
代码与工程

1:52:23 we are going to use this update where we you see that I'm iterating over I've arranged the grads
我们用这个更新的 来了解我正在循环 我安排了研究生

1:52:30 to be in the same order as the parameters and I'm zipping them up the gradients and the
和参数的顺序相同 我把它们拉上梯度和

1:52:34 parameters into p and grad and then here I'm going to step with just the grad that we
升入p和grad参数 然后在这里我将步进

1:52:39 derived manually so the last piece is that none of this now requires gradients from pytorch and so
手动从中产生,所以最后一块是,这些现在不需要从烧火机等

pytorch
PyTorch PyTorch
代码与工程

1:52:49 one thing you can do here is you can do with torsion no grad and offset this whole code block
唯一能做的一件事就是 你可以用无研究生的 火炬来抵消整个代码块

1:52:58 and really what you're saying is you're telling pytorch that hey I'm not going to call backward
你说的是 你在告诉皮托奇 我不会叫后退

pytorch
PyTorch PyTorch
代码与工程

1:53:01 on any of this and this allows pytorch to be a bit more efficient with all of it
使火炬能更高效地使用它

pytorch
PyTorch PyTorch
代码与工程

1:53:05 and then we should be able to just run this and it's running and you see that loss of backward is
然后我们应该能够运行它, 它正在运行 你看到,失去的落后是

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:53:16 commented out and we're optimizing so we're going to leave this run and hopefully we get a good result
评论出来,我们优化了 所以我们要离开这个跑跑 希望我们能有好结果

1:53:25 okay so I allowed the neural net to finish optimization then here I calibrate the
所以我允许神经网完成优化 然后在这里我校准

neural net
神经网络 神经网络
神经网络基础

1:53:30 bachelor of parameters because I did not keep track of the running mean and very variance in
参数的学士学位

1:53:35 their training loop then here I ran the loss and you see that we actually obtained a pretty good
他们的训练循环 然后在这里我输了 你瞧 我们实际上得到了一个不错的

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:53:40 loss very similar to what we've achieved before and then here I'm sampling from the model and we
损失与我们之前和这里所实现的相似 我从模型中抽样

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:53:46 see some of the name like gibberish that we're sort of used to so basically the model worked
我们看到一些名字,比如胡胡说八道 我们习惯了这样基本上这个模型工作

1:53:50 and samples pretty decent results compared to what we're used to so everything is the same
和样本的样本 与我们习惯相比, 相当不错的结果

1:53:56 but of course the big deal is that we did not use lots of backward we did not use pytorch autograd
但最重要的当然是 我们没有使用很多落后的 而不是用火炬自动升级

pytorch
PyTorch PyTorch
代码与工程

autograd
自动微分 自动微分
反向传播与梯度

1:54:01 and we estimated our gradients ourselves by hand and so hopefully you're looking at this the backward
我们用手对梯度进行估算 所以希望你们能看到

1:54:07 pass of this neural net and you're thinking to yourself actually that's not too complicated
神经网的传经 你对自己的想法 其实并不复杂

neural net
神经网络 神经网络
神经网络基础

1:54:13 each one of these layers is like three lines of code or something like that
每个层中每个层 都像三行代码之类的

1:54:16 and most of it is fairly straightforward potentially with the notable exception of
其中大部分可能相当直率,但值得注意的是,

1:54:20 the batch normalization backward pass otherwise it's pretty good okay and that's everything I
批次正常化后退通过 否则它很好,没关系 这就是我的一切

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

backward pass
反向传播 反向传播
神经网络基础

1:54:25 wanted to cover for this lecture so hopefully you found this interesting and what I liked about it
想要为这次演讲做掩护 所以希望你发现这个有趣的东西

1:54:30 honestly is that it gave us a very nice diversity of layers to back propagate through and I think it
诚实地说,它给了我们一个非常好的多层 来回传播,我想它

1:54:37 gives a pretty nice and comprehensive sense of how these backward passes are implemented and how
能够很好地全面了解这些落后通行证是如何实施的,以及如何执行

1:54:41 they work and you'd be able to derive them yourself but of course in practice you probably
工作,你也可以自己画出来 但当然在实际中 你可能

1:54:45 don't want to and you want to use the pytorch autograd but hopefully you have some intuition
不想,你想用火炉自化 但希望你有一些直觉

pytorch
PyTorch PyTorch
代码与工程

autograd
自动微分 自动微分
反向传播与梯度

1:54:49 about how gradients flow backwards through the neural net starting at the loss and how they
梯度如何从损失开始向后流穿过神经网,以及它们是如何向后流的

neural net
神经网络 神经网络
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:54:54 flow through all the variables and all the intermediate results and if you understood a good
在所有变量和中间结果中,如果您了解一个好的

1:54:59 chunk of it and if you have a sense of that then you can count yourself as one of these
部分,如果你有这种感觉的话 你可以把自己算作其中之一

1:55:03 buff doji's on the left instead of the doji's on the right here now in the next lecture we're
杜吉在左边,而不是杜吉 在右在这里 现在在下一场讲座上

1:55:09 actually going to go to recurrent neural nets ls dms and all the other variants of Arnes and
和所有其他的阿恩斯变种

1:55:15 we're going to start to complexify the architecture and start to achieve better
我们开始把建筑结构复杂化 并开始实现更好的

1:55:18 a lot of black limits and so I'm really looking forward to that and I'll see you then

黑底界限很多 所以我真的很期待这个 到时见

点击任意句子从该处播放 · 空格 播放/暂停 · T 中文 · C 连播 · ↑ ↓ 上下句 · Esc 停止

字幕由 ASR 转录并经 AI 校正重译，术语释义仅供参考。